

Conception et Programmation Orientée-objet Avancées

2019 – 2020
Département Informatique
IUT de Nantes



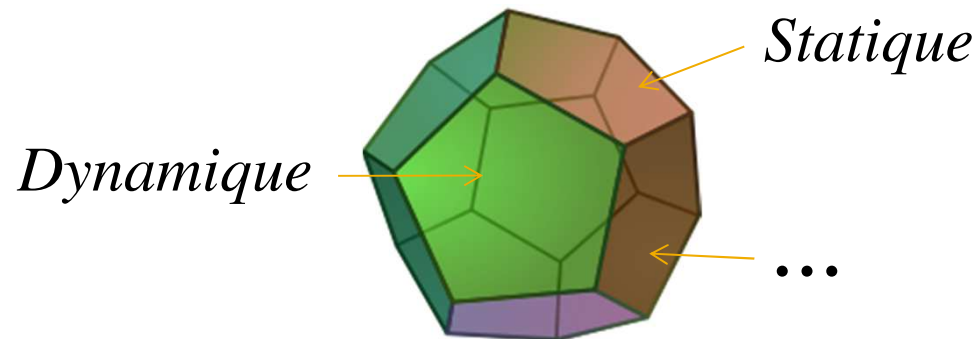
Dalila TAMZALIT ©

CPOA selon le PPN

- **Objectif:** « *produire une **conception détaillée** en appliquant des **patrons de conception**, la mettre en œuvre en utilisant des **bonnes pratiques** de programmation orientée objet* »
- **Compétences visées :**
 - *Analyse d'une solution informatique*
 - *Conception technique d'une solution Informatique*
 - *Réalisation d'une solution informatique*

Rappel : diviser pour mieux régner

- Différentes facettes d'un système à automatiser
(Résultat : un joyau ou une bombe à retardement)



- Chaque facette représente un aspect du système
 - Dynamique
 - Statique
 - ...

Modéliser, Modèle

- Modéliser :
 - Maîtriser la complexité en s'abstrayant du « bruit »
 - Abstraire la réalité pour comprendre le système à réaliser/réalisé.
- Une seule « vue » du système n'est pas suffisante
- Le modèle permet de :
 - spécifier le système à réaliser/réalisé
 - valider le modèle vis-à-vis des clients
 - fournir un guide pour la construction du système
 - documenter le système et les décisions prises

Cycles de développement - Méthodologies

- Cycles de développement logiciels
 - Modèle en cascade
 - Cycle en V
 - Cycle en spirale
 - Cycle semi-itératif
 - Cycle itératif
- Méthodologies de développement
 - eXtreme Programming
 - Scrum
 - RUP
 - Merise
 - SADT



UML « juste » pour modéliser

- Ne dit pas comment faire :
 - ne donne pas de directives quant aux étapes à suivre ni comment faire.
- Permet de produire des modèles :
 - Visualiser
 - Spécifier
 - Construire
 - Documenter
- Version actuelle : 2.5.1 (décembre 2017)

14 diagrammes UML

- **Diagrammes structurels ou diagrammes statiques (*UML Structure*)**

1. diagramme de classes (*Class diagram*)
2. diagramme d'objets (*Object diagram*)
3. diagramme de composants (*Component diagram*)
4. diagramme de déploiement (*Deployment diagram*)
5. diagramme de paquetages (*Package diagram*)
6. diagramme de structures composites (*Composite structure diagram*)
7. diagramme de profil (*Profil diagram*)

- **Diagrammes comportementaux ou dynamiques (*UML Behavior*)**

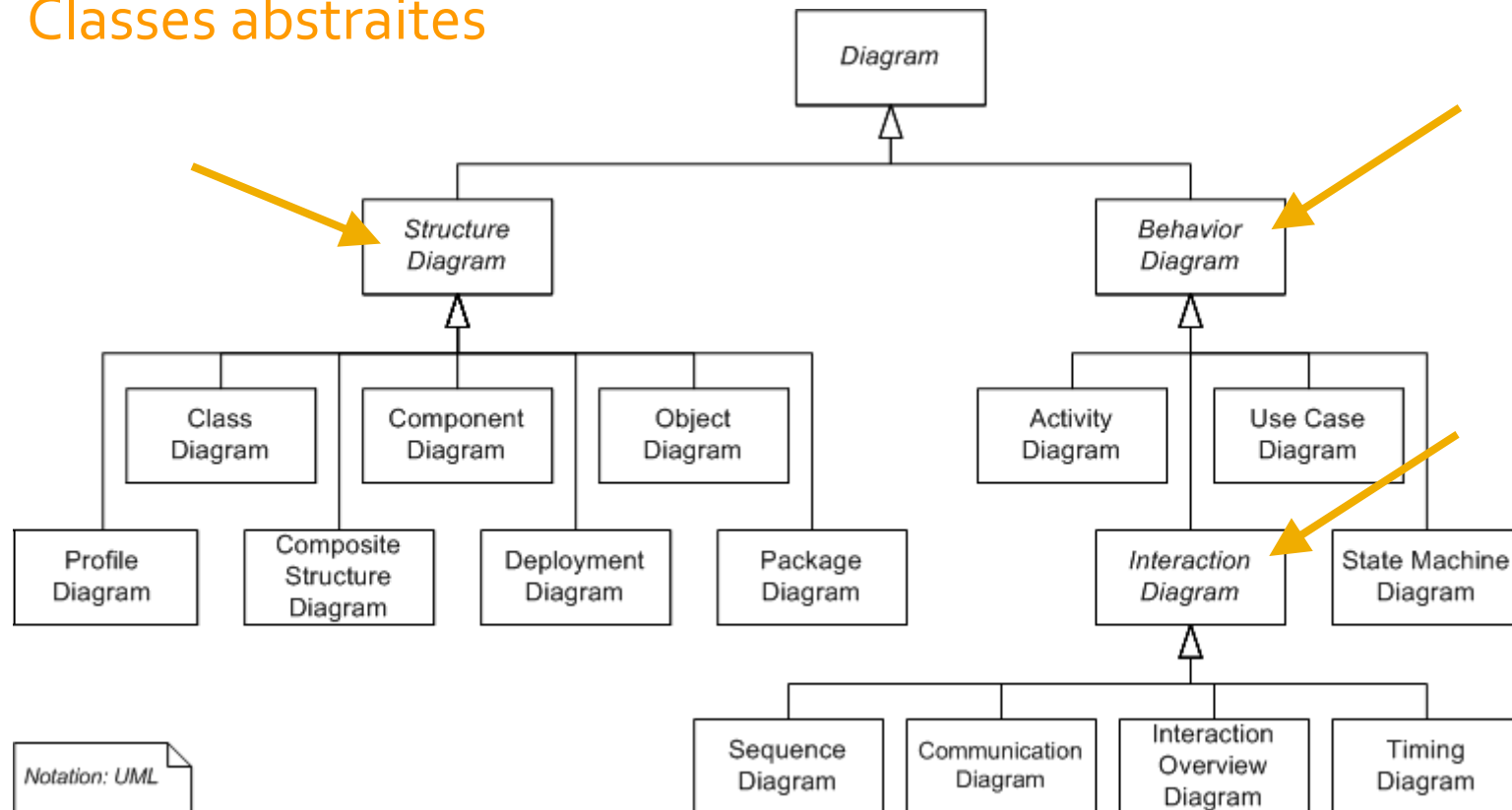
8. diagramme de cas d'utilisation (*Use case diagram*)
9. diagramme d'activités (*Activity diagram*)
10. diagramme d'états-transitions (*State machine diagram*)

- **Diagrammes d'interaction (*Interaction diagram*)**

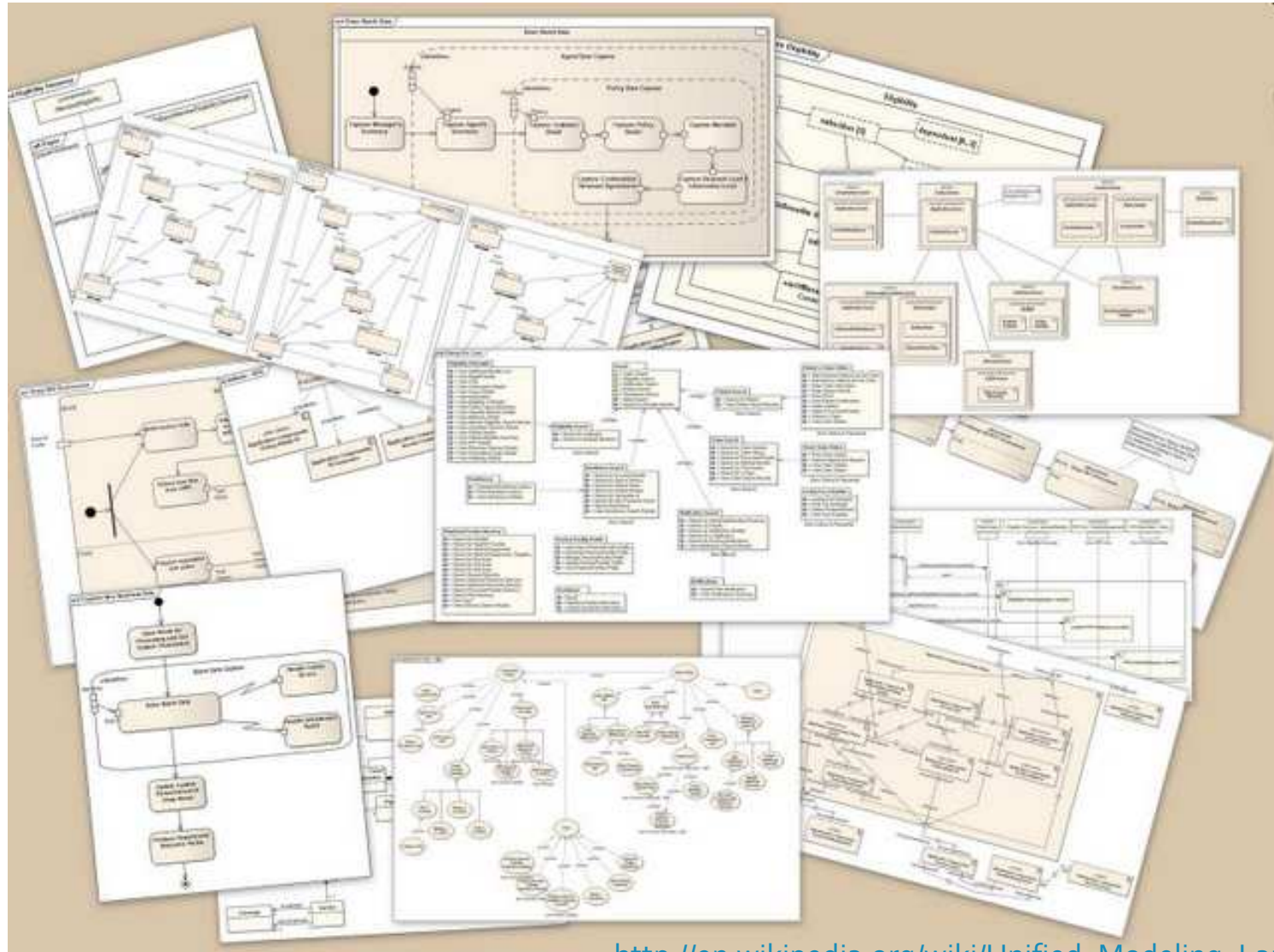
11. diagramme de séquence (*Sequence diagram*)
12. diagramme de communication (*Communication diagram*)
13. diagramme global d'interaction (*Interaction overview diagram*)
14. diagramme de temps (*Timing diagram*)

14 diagrammes d'UML

Classes abstraites

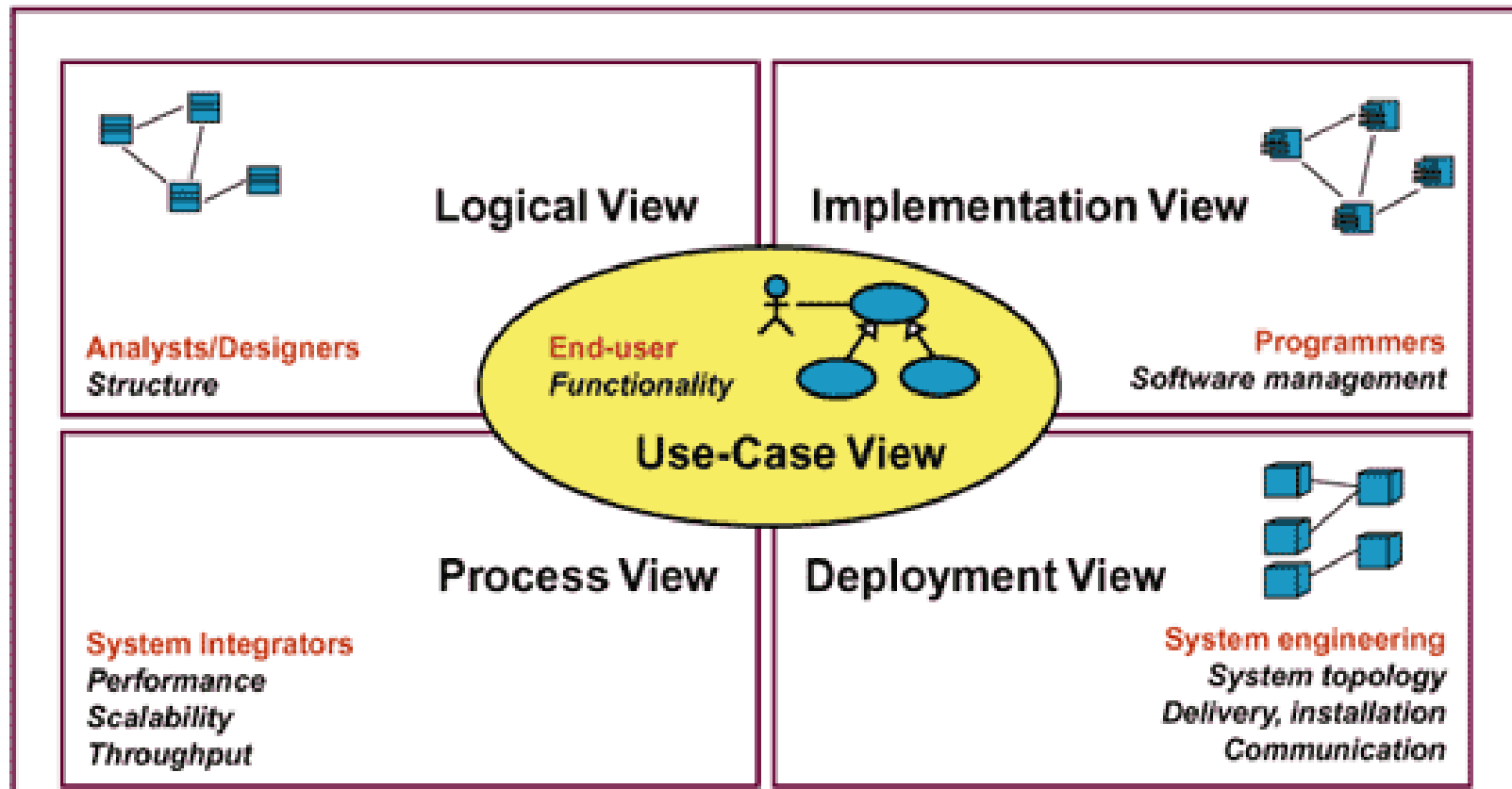


Diagrammes UML d'un système



http://en.wikipedia.org/wiki/Unified_Modeling_Language

Le modèle « 4 + 1 vues »_{Ph. Kruchten*}



Le modèle d'architecture de *Kruchten*, dit *modèle des 4 + 1 vues*, est adopté dans l'*Unified Process*. La vue des cas d'utilisation établit le lien et dirige la création des autres diagrammes d'architecture. * Philippe B. Kruchten, *The 4+1 View Model of Architecture*, IEEE Software, novembre 1995.

Le modèle « 4 + 1 vues » Ph. Kruchten

Vue logique : principale, souvent unique, de l'architecture.

Statique : *Diagrammes de classes* et *Diagrammes de composants*.

Dynamique : *Diagrammes d'interactions* (séquences, communication) et *d'états*.

Vue des processus : interactions entre différents processus et allocation des objets (fil d'exécution, tâches, paramètres, ...).

Diagrammes d'activités, *séquences*, *diagrammes de communication* ou *d'états-transitions*.

Vue de développement (réalisation) : vue de l'organisation des composants.

Diagrammes de composants.

Vue des cas d'utilisation : analyse du système par les utilisateurs.

Diagramme de cas d'utilisation

Vue de déploiement (physique) : déploiement, noeuds, modules et différentes contraintes (géographiques, bandes passantes, QoS...).

Diagrammes de composants et *diagrammes de déploiement*.

Phases clés : recueil des besoins

- Compréhension du système
- Comprendre et structurer les besoins
- Définir les frontières du système
- Identifier les principales fonctionnalités métier du système

Phases clés : Analyse

- Autre terminologie : conception architecturale, conception générale, analyse fonctionnelle
- Utilisation et utilité :
 - Outil de dialogues entre clients et utilisateurs
 - Comprendre les besoins métiers et les décrire
 - Extraire les modèles de l'expression des besoins
 - Uniquement les entités du domaine (métier)
 - Support pour la conception, l'implémentation et la maintenance
 - Description métier de ce que doit faire le système et comment il le fait **sans** la solution technique (trop tôt)

Phases clés : Conception

- Autre terminologie : conception détaillée
- Utilisation et utilité :
 - Destinée aux développeurs
 - Complète ou raffine les modèles de l'analyse avec des éléments et choix techniques
 - Prépare l'implémentation
 - Décrit la solution : comment le problème est résolu
 - Organiser le développement, identifier les dépendances entre modules, la configuration, la gestion des versions...
 - Choix langages, définir les schémas de bases de données
 - Support à la maintenance

Avant d'aller plus loin...

- ... Reprenons les diagrammes suivants pour introduire des concepts avancés :
 - Diagramme de cas d'utilisation
 - Diagramme de classes

Diagramme de cas d'utilisation

- Rappels
- Concepts avancés

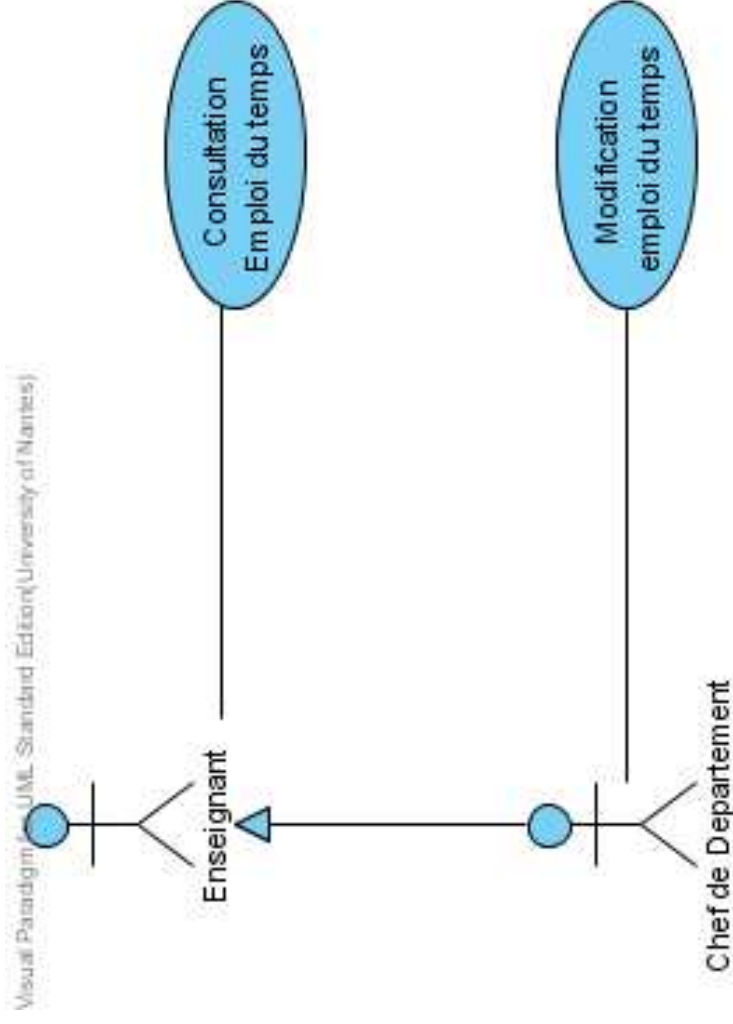
Diagramme cas d'utilisation

- Rappel de la relation possible entre acteurs : généralisation/spécialisation
- Rappels des relations entre cas d'utilisation :
 - « *includes* »
 - Généralisation/spécialisation
- Autre relation : « *extends* »

Types d'acteurs

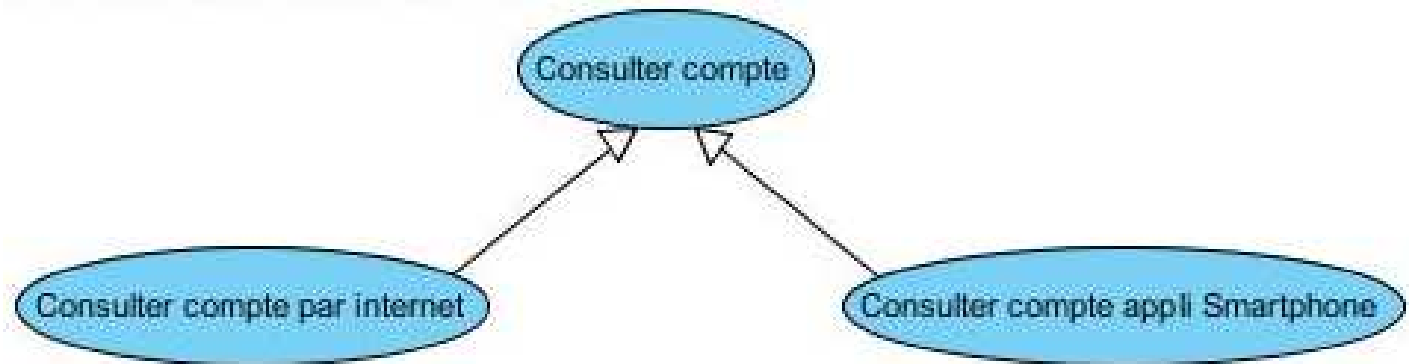
- L'acteur qui déclenche un cas d'utilisation :
 - Stéréotype : « Principal ».
 - Partie prenante qui demande au système d'offrir un de ses services.
- L'acteur qui fournit un service :
 - Stéréotype : « Secondaire ».
 - Partie externe sollicitée par le système (une imprimante, un service web, une banque, un système d'information...)

Généralisation entre acteurs



Généralisation/spécialisation CU

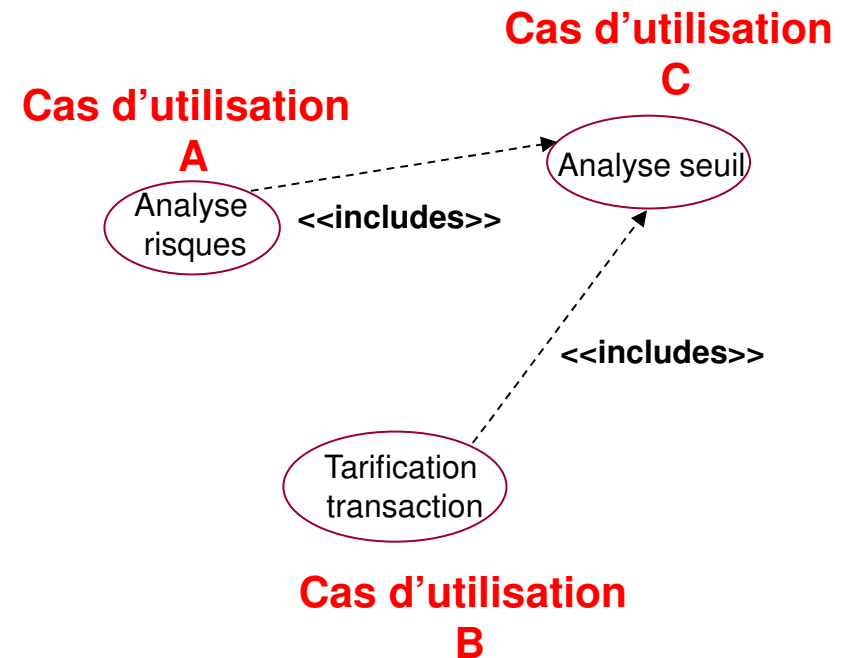
- Spécialisation de certaines actions du cas d'utilisation d'origine



- Le premier spécialise certaines étapes de la séquence d'actions du second en les remplaçant par des « versions » plus spécialisées.
- Toutes les étapes du cas d'utilisation original doivent être exécutées, certaines étant remplacées par des « versions » plus spécialisées.

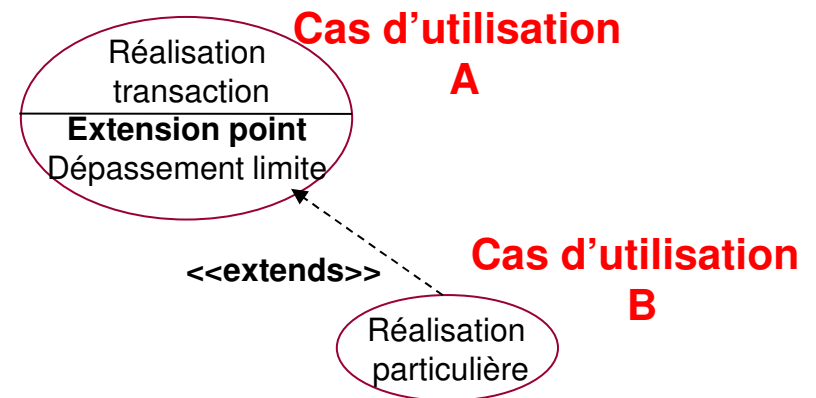
« includes »

- La relation <<includes>> permet à la fonctionnalité commune des cas d'utilisation A et B d'être décrite par un cas d'utilisation C.
- C est toujours nécessaire pour que A et B puisse s'exécuter.



« extends »

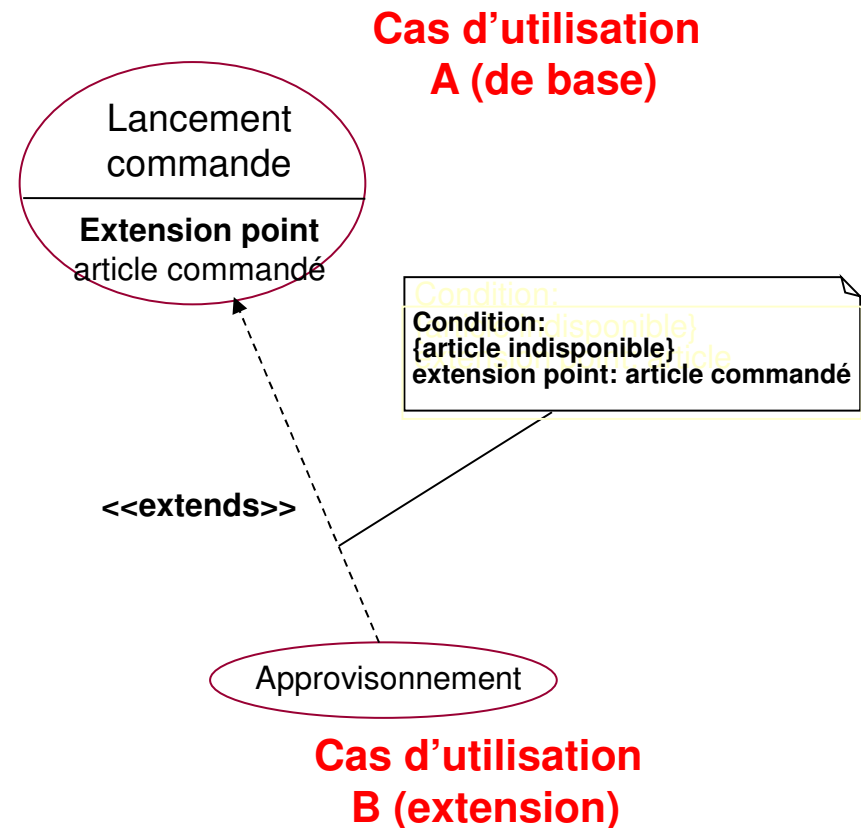
- Ajout de fonctionnalité facultative
- A: cas d'utilisation de base
- B: cas d'utilisation spécifique qui étend et complète le cas d'utilisation A
- A peut être exécuté seul ou avec ses extensions



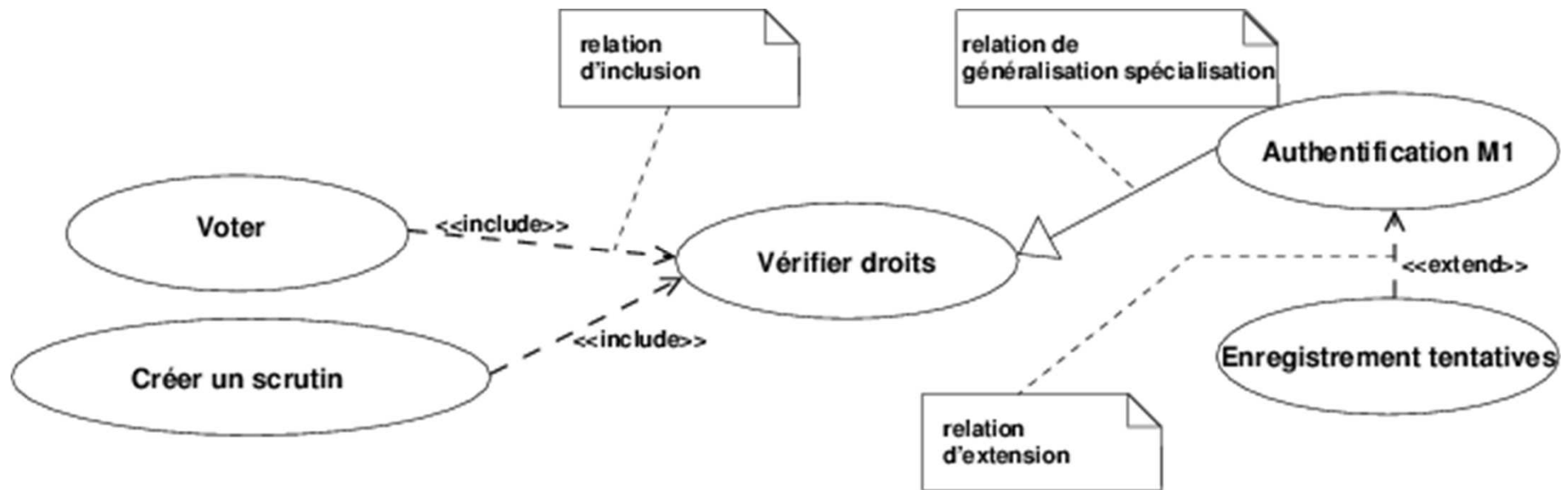
- Les points d'extension indiquent les emplacements auxquels le cas d'utilisation A est complété

« *extends* »

- Pour ajouter une extension, il faut qu'une condition soit remplie.
- Une condition peut être spécifiée sous forme de note ou de commentaire attaché à la relation << *extends* >>



Exemple



<http://www-inf.it-sudparis.eu/COURS/CSC4002/EnLigne/Cours/CoursUML/>

Pièges classiques

- Modéliser un processus : pas de chronologie/temporalité dans un diagramme de cas d'utilisation.
- Mauvaise délimitation des frontières du système : acteurs toujours en dehors des frontières.
- Problème de décomposition : un cas d'utilisation effectue un service de bout en bout. Autrement penser scénarios.

Diagramme de classes

- Rappels
- Concepts avancés

Diagrammes classes/d'objets

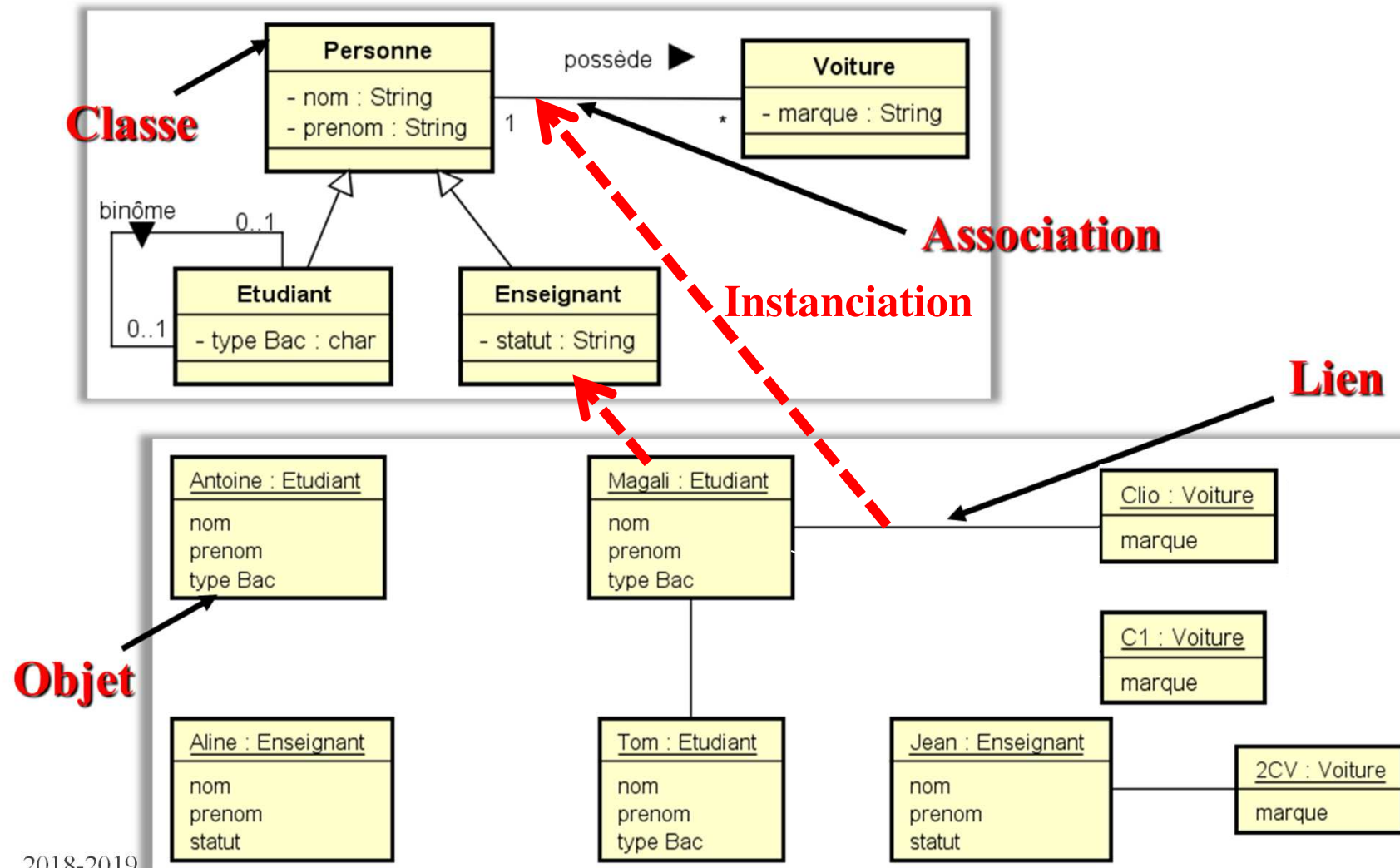


Diagramme d'objets : formalismes

Antoine

Antoine : Etudiant

: Etudiant

Antoine

nom = " Dupont "
prenom = " Antoine "
type Bac = " S "

Antoine : Etudiant

nom = " Dupont "
prenom = " Antoine "
type Bac = " S "

: Etudiant

nom = " Dupont "
prenom = " Antoine "
type Bac = " S "

Quid des opérations ?

Diagramme de classes : formalismes

Personne

Visibilité	Symbole
public	+
private	-
protected	#
package	~

Personne
nom prenom adresse date naissance
recupererAdresse() : void

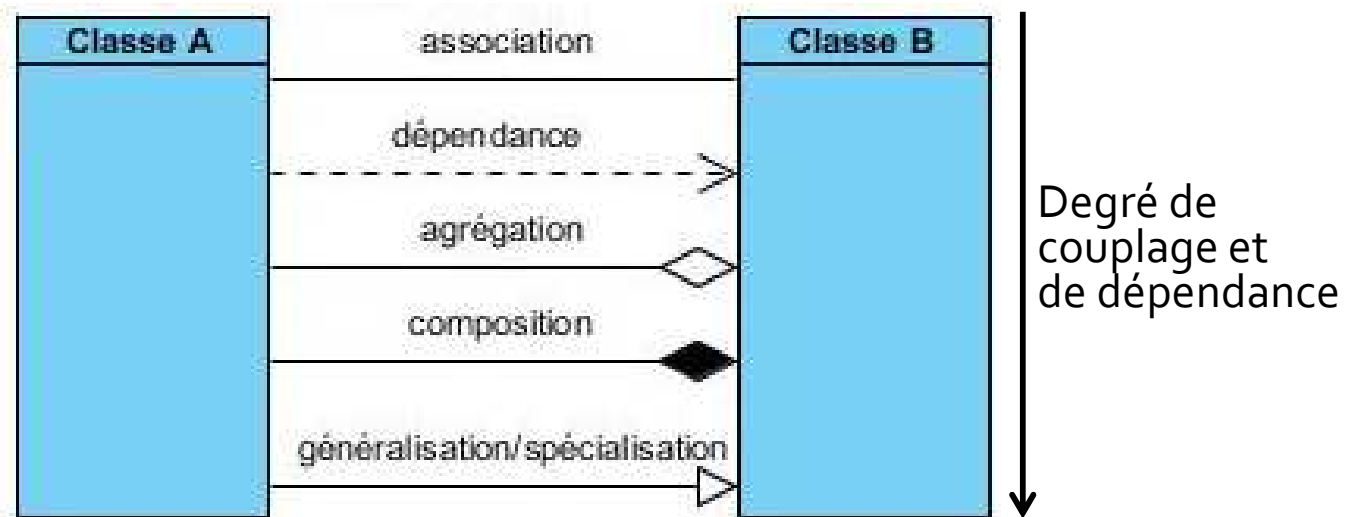
Personne
- nom : String - prenom : String - adresse : String - date naissance : String

Personne
- nom : String - prenom : String - adresse : String - date naissance : String
+ recupererAdresse() : void

Personne
- nom : String - prenom : String - adresse : String - date naissance : String - / age : int
+ recupererAdresse() : void

Associations & Relations

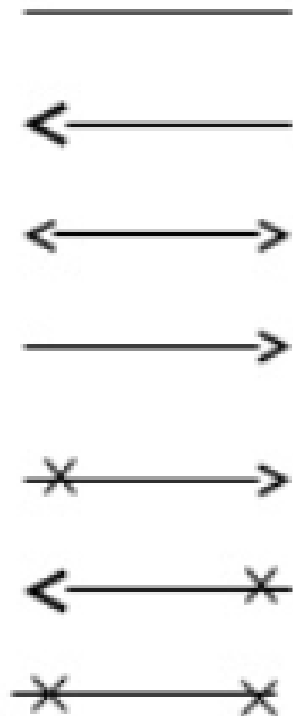
- Au cœur des phases d'analyse et de conception
- **Différents types de relations :**
 - entre classes :



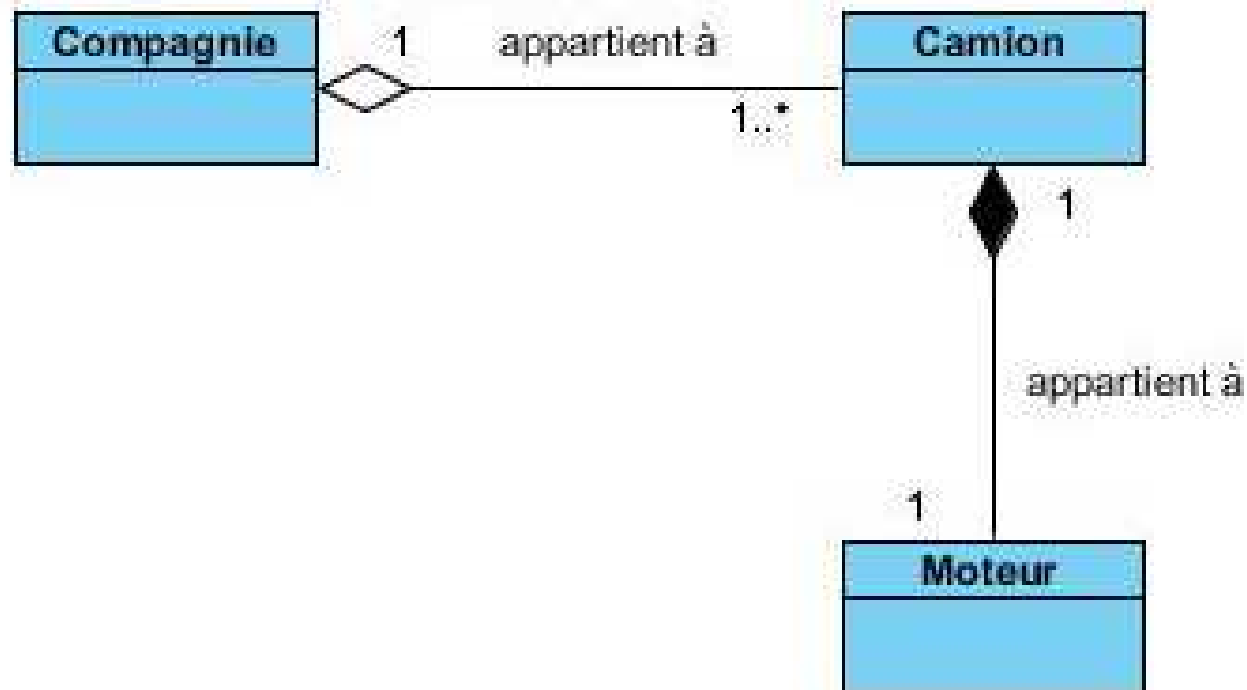
- **entre classe et interface :** implémentation & association
- **entre interfaces :** généralisation

Association et navigation

- En cas de doute, ne mettre aucune flèche



Agrégation et Composition



Agrégation

- Une association exprimant un couplage fort lié à une relation de subordination : fort couplage logique
- Bidirectionnelle et asymétrique.
- **Attention :**
 - Un élément agrégé peut être lié à d'autres classes
 - La suppression de l'ensemble n'entraîne pas celle de l'élément
- **Exemple :** un ouvrage (*agrégat*) et ses exemplaires (*composant*).

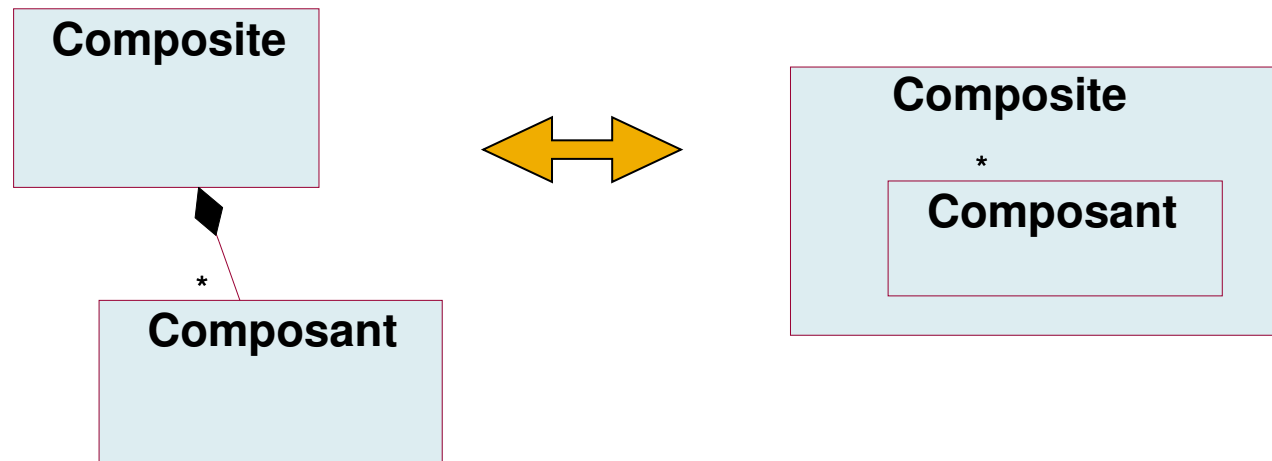


Composition

- La composition est
 - une agrégation forte
 - lie les cycles de vie entre le composé ou composite (ensemble) et les composants (éléments)
- Les règles supplémentaires obligatoires pour la composition sont les suivantes :
 - la suppression du composé entraîne la suppression des composants
 - les composants sont créés par le composé
 - Un composant est lié à un composé unique

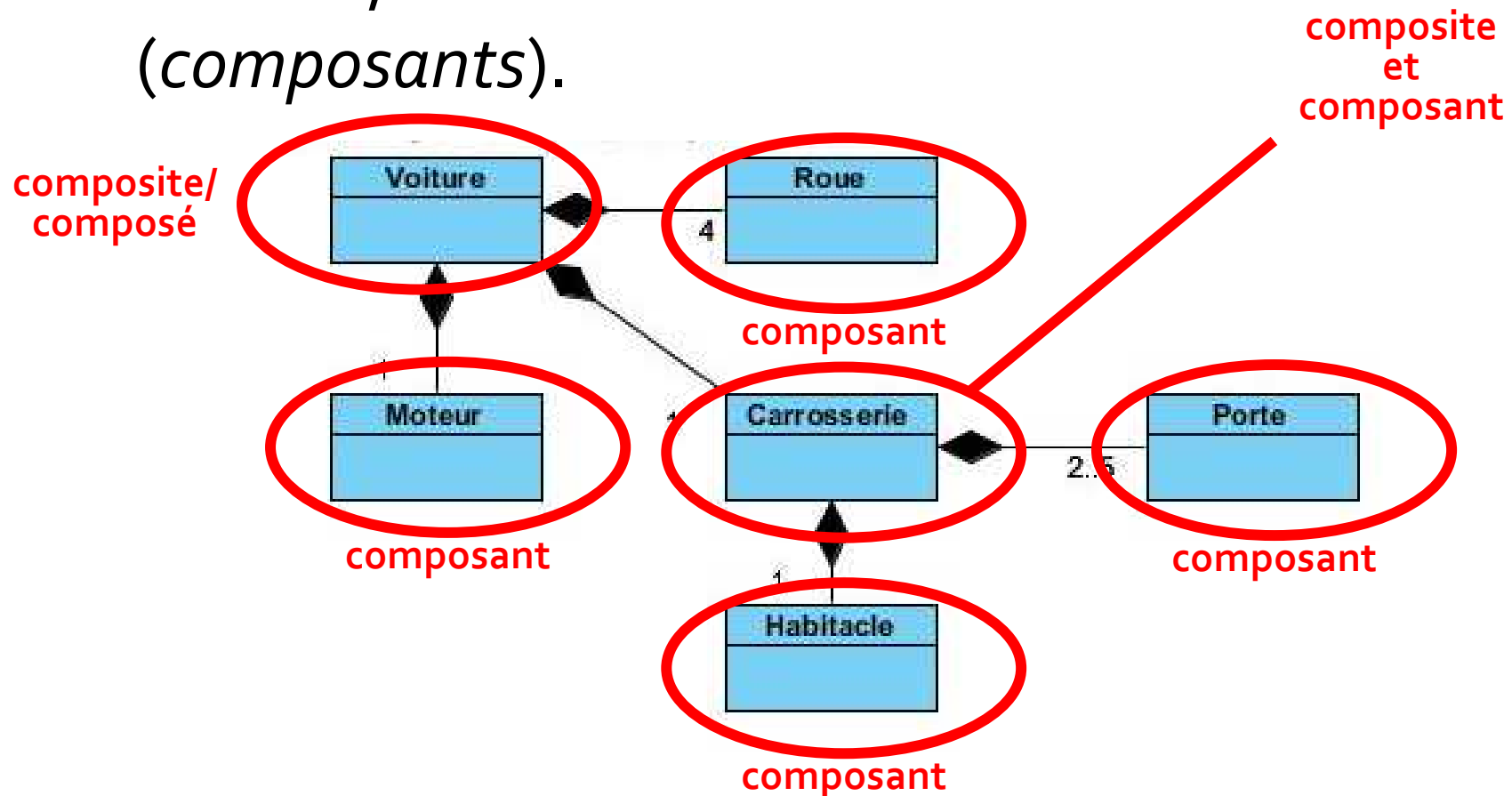
Composition

- Exprime souvent une composition physique.
- UML offre deux possibilités de représentation :

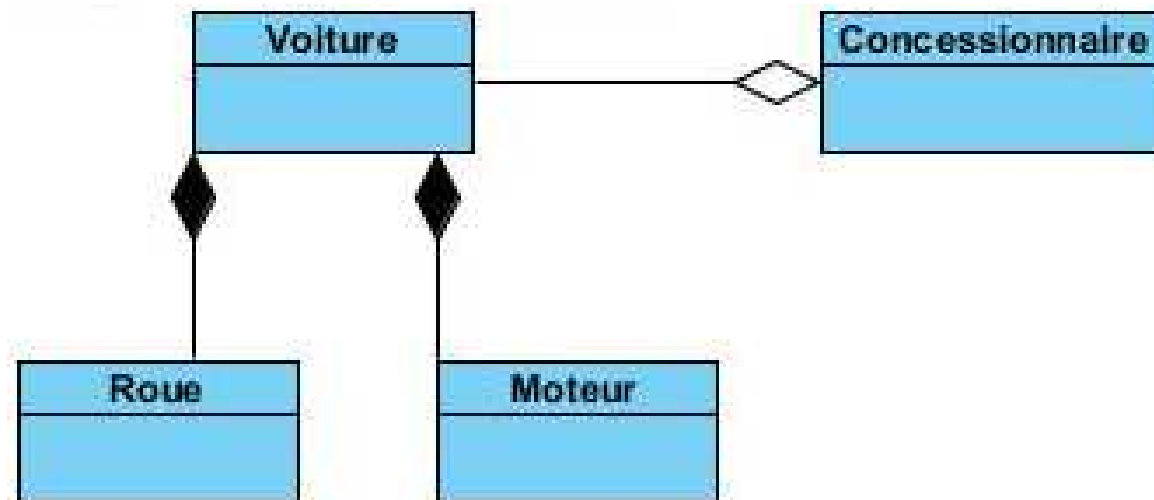


Composition

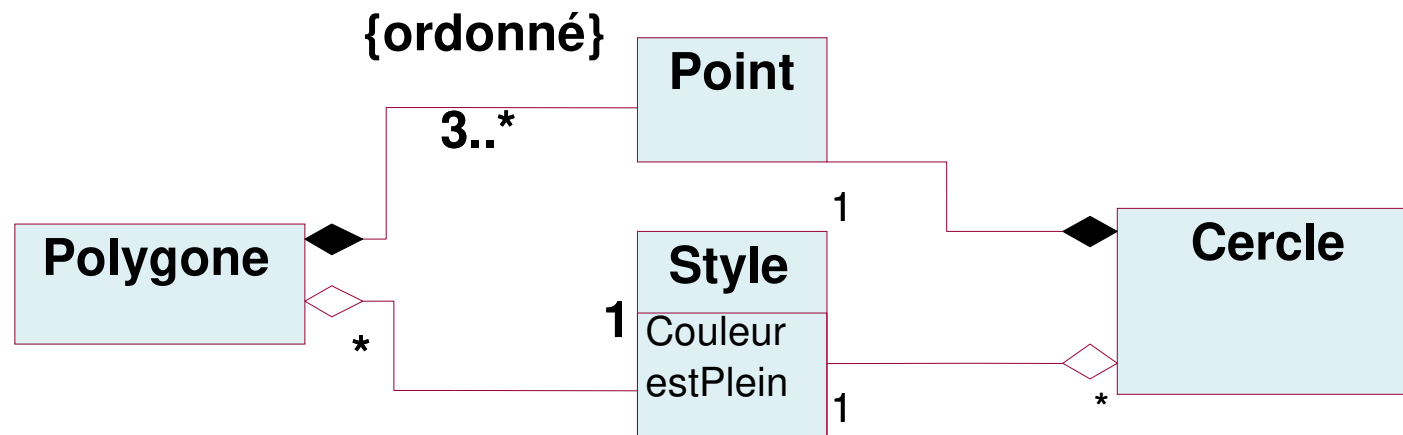
- Exemple : une voiture (*composite*) et son moteur, sa carrosserie et ses roues (*composants*).



Agrégation et composition



Agrégation et Composition



- Un Polygone ainsi qu'un Cercle sont composés de points :
 - La destruction du composite implique la destruction de la composition.
 - La destruction du composant implique celle du composite.
- Un Style peut concerner plusieurs Polygones et/ou Cercles :
 - Style est indépendant de l'existence d'agrégats.
 - Sa modification implique forcément celle de ses agrégats.

Agrégation & composition vers le Code

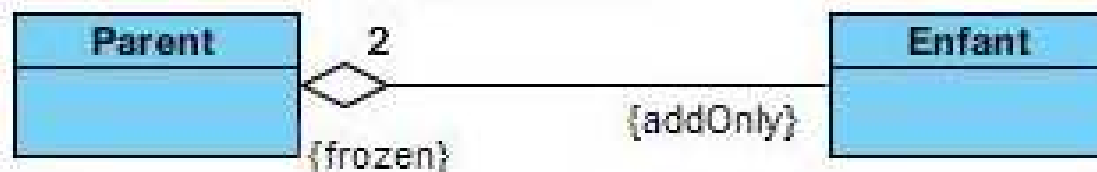
- Agrégation = Association
 - Règles de passage au code identiques,
- Composition = association + dépendance forte de la vie des composants vis-à-vis de leur composite
 - Règles de passage au code identique à une association
 - **+ prévoir un destructeur !**

D'autres contraintes entre associations

- *{addOnly}* : autorise l'ajout de nouveaux objets mais pas leur suppression ni leur mise à jour.



- *{Frozen}* : interdit l'ajout, la suppression ou la mise à jour des liens d'un objet vers les objets de la classe associée.



Stéréotype

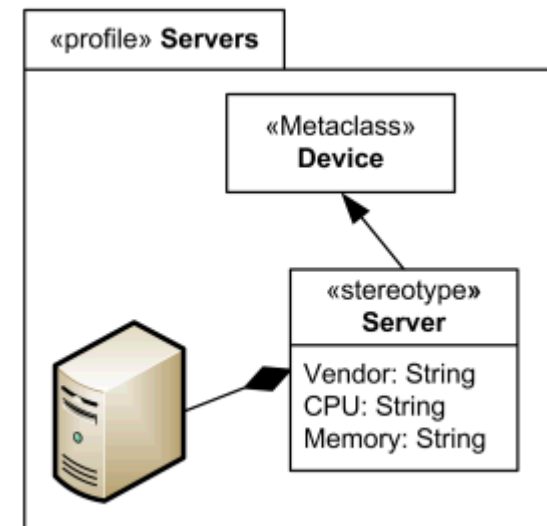
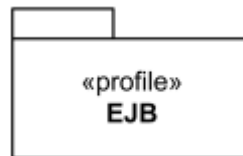
- Principal mécanisme d'extension d'UML.
- Permet de construire de nouveaux types, par exemple des objets affichables.
- Généralement représenté par un texte entre guillemets

■ Exemple `<<Objets Dessinables>>`
et une icône associée.



- Stéréotypes de classe, d'association ou de généralisation.

Exemples stéréotypes



Qualification

- Une association peut être qualifiée.
- Restriction qui consiste à sélectionner un sous-ensemble d'objets parmi l'ensemble des objets qui participent à une association
 - Restriction de la multiplicité



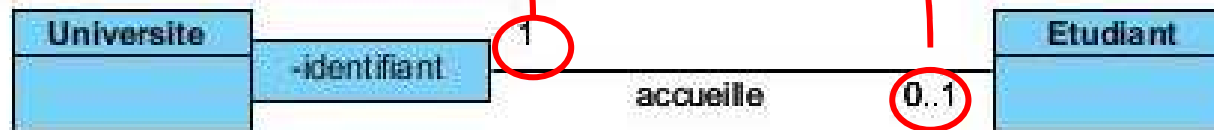
- La paire (instance de la classe A, valeur de la qualification) identifie un sous-ensemble des instances de la classe B.
- La clé est représentée sur le rôle de la classe de départ, dans un rectangle.

Exemples qualification

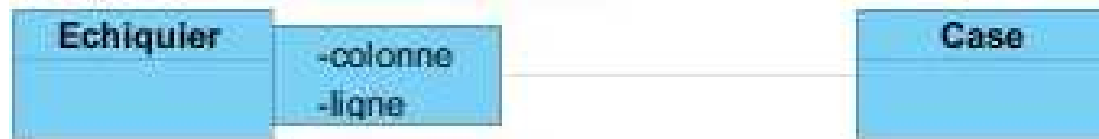
- Une université peut accueillir plusieurs étudiants et un étudiant peut être inscrit à plusieurs universités.



- Un étudiant a un identifiant unique dans une université qui l'accueille



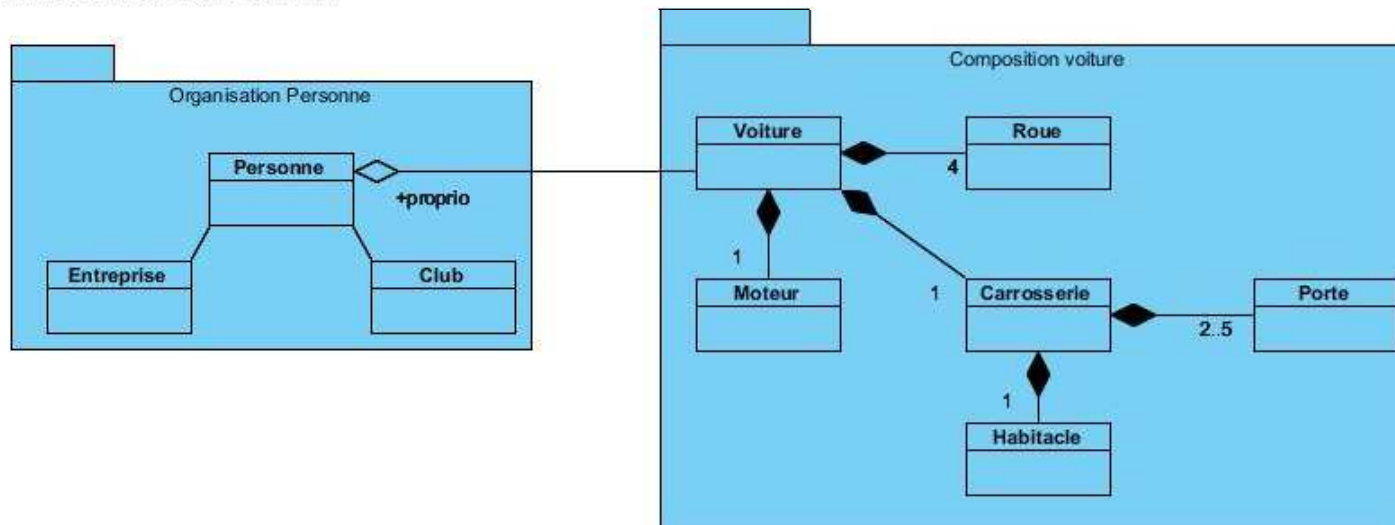
- Clés multiples



Paquetages

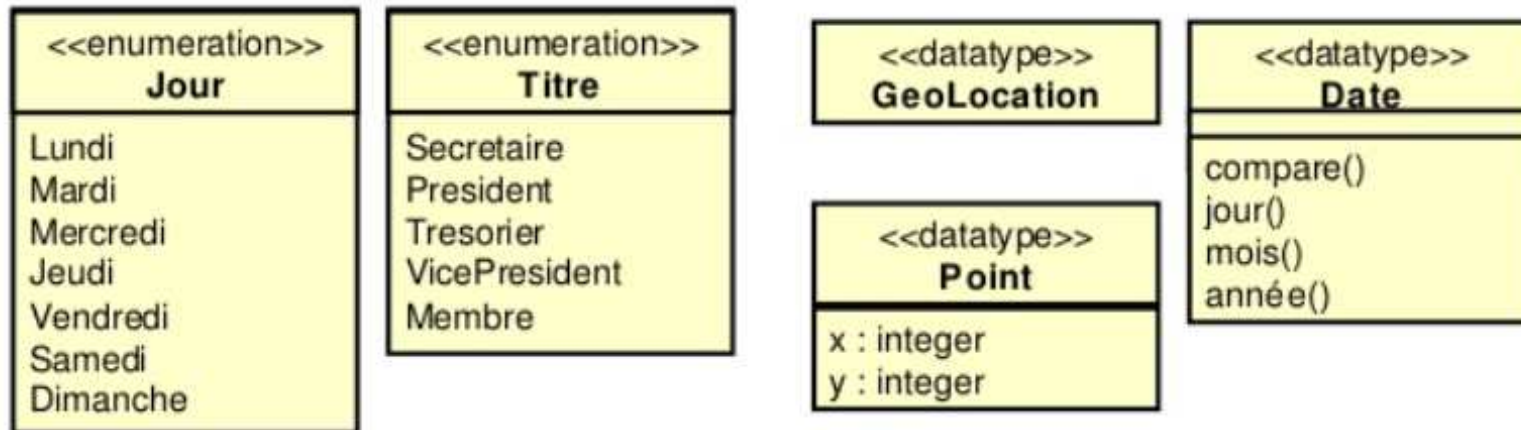
- Un paquetage (*package*) peut regrouper des classes, des interfaces et des paquetages.

Visual Paradigm Standard Edition (University of Nantes)



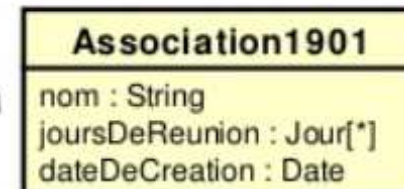
- Relations entre paquetages :
 - Relations classiques entre classes
 - Relations d'import

Enumération et types de données



- Utilisable comme type d'attributs
- Valeurs (pas d'identité)
- Type de données typiquement définis dans des « librairies »

Exemple
d'utilisation



Diagrammes de classes

- d'analyse
- de conception

Diagramme de classes d'analyse

- Classes du domaine métier uniquement
- Use cases dans le langage du client mais diagramme d'analyse dans le langage du concepteur
- Objectif :
 - clarifier le domaine métier, se familiariser avec
 - pas de classes techniques
- Diagramme de classes d'analyse :
 - classes, attributs, associations
 - pas d'opérations
- Démarche usuelle
 - diagramme de classes par use case
 - recouper les diagrammes de classes

Construction du DCA

- Recenser les objets métier dans :
 - les documents de définition des besoins
 - les documents d'analyse
- Noms ou groupes nominaux en classe, objet ou attribut.
- Doute entre attribut ou classe : préférer la classe puis affiner par la suite.
 - Exemple : adresse en attribut de Personne ou en classe associée à Personne ?
- Attention aux faux amis et doublons.
- Eviter les classes fonctionnelles qui font du traitement :
 - gestionnaire du planning, décodeur de message ...

Construction du DCA

- Identifier les classes d'objets
 - puis garder les bonnes classes
 - Constitution du dictionnaire de données
- Identifier les associations
 - puis garder les bonnes associations
- Identifier les attributs
 - puis garder les bons attributs
- Raffiner au moyen de l'héritage
 - Généralisations et raffinages
- Itérer la modélisation
- Grouper les classes en modules/paquetages

Diagramme de classes d 'analyse

- Démarche

- Dictionnaire des données/glossaire
- Les classes (noms communs)
- Les objets (noms propres ou nom référencés)
- Les attributs (données simples qui qualifient une classe ou un objet)
- Les éléments qui sont étrangers à notre problème ou qui n'y ont pas de rôle réel sont omis

Diagramme de classes de conception

- Cette activité fait suite à la phase d'analyse,
- Diagramme contenant les classes qui seront développées. Aboutissement du travail d'analyse,
- Point(s) de départ :
 - Diagramme de classes d'analyse
 - Diagrammes d'interactions

Diagramme de classes de conception

- Cette activité consiste à :
 - décrire précisément les opérations et les attributs des classes de conception
 - décrire les relations auxquelles elle prend part :
Vérifier/ajouter/supprimer des associations/dépendances
 - décrire ses états imposés (diagramme d'états)
 - expliciter ses méthodes (qui réalisent les opérations) et la réalisation correcte de ses interfaces
 - exprimer les exigences sur son implémentation
- Eventuellement des classes du DCA disparaîtront, des classes apparaîtront.

DCA Vs DCC

- DCA :
 - Uniquement les classes métiers (du domaine)
 - Se limite aux attributs uniquement et aux associations
 - Utilisation des diagrammes de séquences système (boîte noire) et/ou diagrammes de collaboration
- DCC :
 - Ajout des opérations en se basant sur les diagrammes dynamiques et d'interactions
 - Classes « techniques » (IHM, contrôleur, énumération...)
 - Application des patterns
 - Utilisation des diagrammes de séquences détaillés (boîte blanche)

DCA Vs DCC

- Le DCC étant un raffinement, il est souvent porteurs de concepts avancés qui n'apparaissent pas (forcément) dans le DCA :
 - Méthodes et leur signature
 - Navigabilité
 - Qualification
 - ...

Design patterns ou Patrons de conception

De bonnes pratiques

Patterns ou patrons

- Un patron est une solution qui pourra être réutilisée et adaptée indéfiniment
 - Inspiration : plans d'architecture
 - Autre analogie : patrons pour la couture
- Il existe des patterns à tous les niveaux du cycle de développement:
 - Patterns processus
 - Patterns d'analyse
 - Patterns de conception
 - Patterns d'architecture
 - Patterns d'implémentation

Introduction aux patterns

- Définition simple :
 - Un pattern :
 - est un couple problème/solution nommé
 - applicable à de nouveaux contextes
 - Généralement, il est accompagné de :
 - conseils sur la manière de l'utiliser
 - ses avantages et inconvénients
- Les patterns ou Patrons
 - sont applicables à l'infini
 - se voient affecter une responsabilité clairement identifiée

Design patterns

- Définir un patron de conception revient au minimum à spécifier :
 - Son **nom** pour le classer et le diffuser
 - Une description du **problème** qu'il traite
 - Une description de la **solution** qu'il apporte
- Toujours : un couple **problème/solution**

Design patterns

- « Design patterns are just convenient ways of reusing object-oriented code between projects and between programmers. The idea behind design patterns is simple – write down and catalogue common interactions between objects that programmers have frequently found useful. »

J. W. Cooper, IBM Thomas J. Watson Research Center, 1998

- Objectifs :
 - Ne pas réinventer la roue
 - Réutiliser des idées de conception
 - Tirer profit de l'expérience d'autrui
 - ...
- Design Patterns - Elements of Reusable Software, Gamma, Helm, Johnson and Vlissides, 1994, dit the Gang of Four ou "GoF" book

Design patterns

- Attention : un DP :
 - n'est pas du code, même s'il peut être implémenté (lorsqu'on arrive à la production du logiciel)
 - est une démarche standardisée de résolution d'un problème déjà rencontré par le passé
 - est un outil de capitalisation de l'expérience de conception et de développement logiciel OO

Patrons du GoF

- Trois familles de patrons de conception :
 - *de construction* : comment instancier et configurer des classes et des objets
 - *structuraux* : comment organiser les classes d'un programme dans une structure plus large (séparant l'interface de l'implémentation)
 - *comportementaux* : comment organiser les objets pour que ceux-ci collaborent (distribution des responsabilités) et comment

Pattern Délégation

```
import java.util.ArrayList;

public class EnsembleNotes {

    /** Un attribut pour l'ensemble des notes */
    private ArrayList notes;

    /** Le constructeur sans paramètre */
    public EnsembleNotes() {
        notes = new ArrayList();
    }

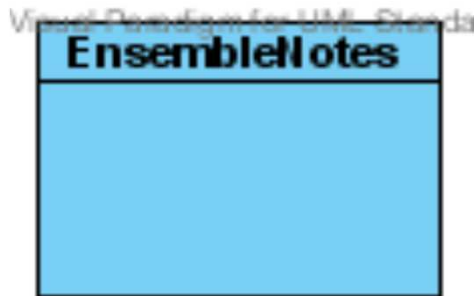
    /** Méthode qui affiche l'ensemble des notes stockées */
    public void listeNotes() {...}

    /** Méthode qui donne le nombre de notes stockées */
    public int nombreNotes() {...}

    /** Méthode qui ajoute une note */
    public void ajoutNotes(Object n) {...}

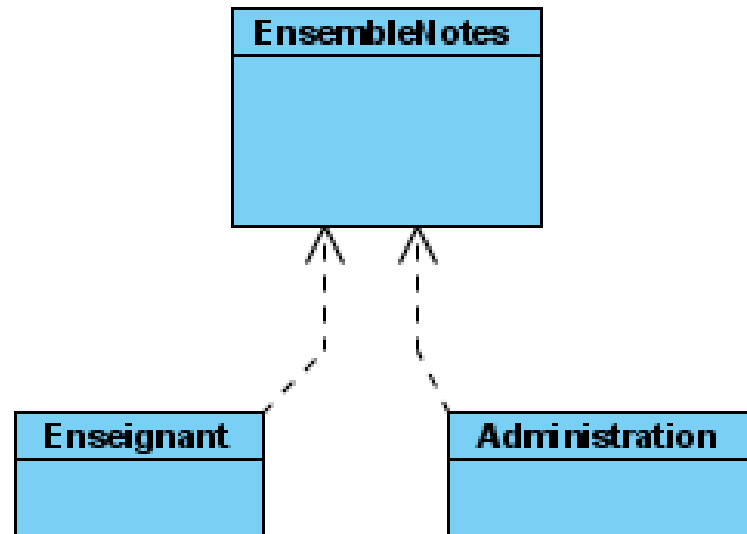
    /** Méthode qui retire une note */
    public void retraitNotes(int index) {...}

}
```



Pattern Délégation

Arrivent des clients pour cette classe



Que savent-ils et que voient-ils de

Class EnsembleNotes

```
java.lang.Object
└─ reutilisationDesignPattern.EnsembleNotes
```

```
public class EnsembleNotes
extends java.lang.Object
```

Constructor Summary

[EnsembleNotes](#) ()
Le constructeur sans paramètre

Method Summary

void	<u>ajoutNotes</u> (java.lang.Object n) Méthode qui ajoute une note
void	<u>listeNotes</u> () Méthode qui affiche l'ensemble des notes stockées
int	<u>nombreNotes</u> () Méthode qui donne le nombre de notes stockées
void	<u>retraitNotes</u> (int index) Méthode qui retire une note

Pattern Délégation

Changement de SDD :
de ArrayList à Vector

```
import java.util.Vector;

public class EnsembleNotes{

    /** Un attribut pour l'ensemble des notes */
    private Vector notes;

    /** Le constructeur sans paramètre */
    public EnsembleNotes_V(){
        notes = new Vector();
    }

    /** Méthode qui affiche l'ensemble des notes stockées */
    public void listeNotes() {...}

    /** Méthode qui donne le nombre de notes stockées */
    public int nombreNotes() {...}

    /** Méthode qui ajoute une note */
    public void ajoutNotes(Object n)
    { notes.addElement(note); }

    /** Méthode qui retire une note */
    public void retraitNotes(int index)
    { notes.removeElementAt(index); }

}
```

Pattern Délégation

Modifications sans conséquences

Visibilité sans changements

Class EnsembleNotes

```
java.lang.Object  
└─reutilisationDesignPattern.EnsembleNotes
```

```
public class EnsembleNotes  
extends java.lang.Object
```

Constructor Summary

[EnsembleNotes](#) ()
Le constructeur sans paramètre

Method Summary

void	<u>ajoutNotes</u> (java.lang.Object n) Méthode qui ajoute une note
void	<u>listeNotes</u> () Méthode qui affiche l'ensemble des notes stockées
int	<u>nombreNotes</u> () Méthode qui donne le nombre de notes stockées
void	<u>retraitNotes</u> (int index) Méthode qui retire une note

Pattern Délégation

```
private ArrayList notes; // ou private Vector notes;  
  
public EnsembleNotes() {  
    notes = new ArrayList(); } // ou notes = new Vector();  
  
public void ajoutNotes(Object n  
    { notes.add(note); } // ou notes.addElement(note);
```

- ❑ Acquisition d'une instance de la classe
- ❑ Délégations des opérations à cette instance
- ❑ Avantages
 - Couplage faible
 - Réutilisation, maintenance, évolution accrues

Pattern Délégation : schéma

```
public class ClasseA{
    //attribut
    private ClasseB classeB;

    //constructeur
    public classeA(){ classeB = new ClasseB(); }

    //méthodes
    public void m(){classeB.m1();} // délégation à m1
    ...
}

public class ClasseB{
    public void m1(){...}
}
```

Proxy – Introduction

- Un objet ordinaire fait le travail qu'il annonce au travers de son interface
- Il peut arriver qu'il ne soit pas en mesure d'assumer cette responsabilité
- Exemples :
 - L'objet met trop de temps à se charger
 - Nécessité d'intercepter des messages qui lui sont adressés
- Un autre objet peut alors prendre cette responsabilité vis-à-vis d'un client en tant qu'intermédiaire

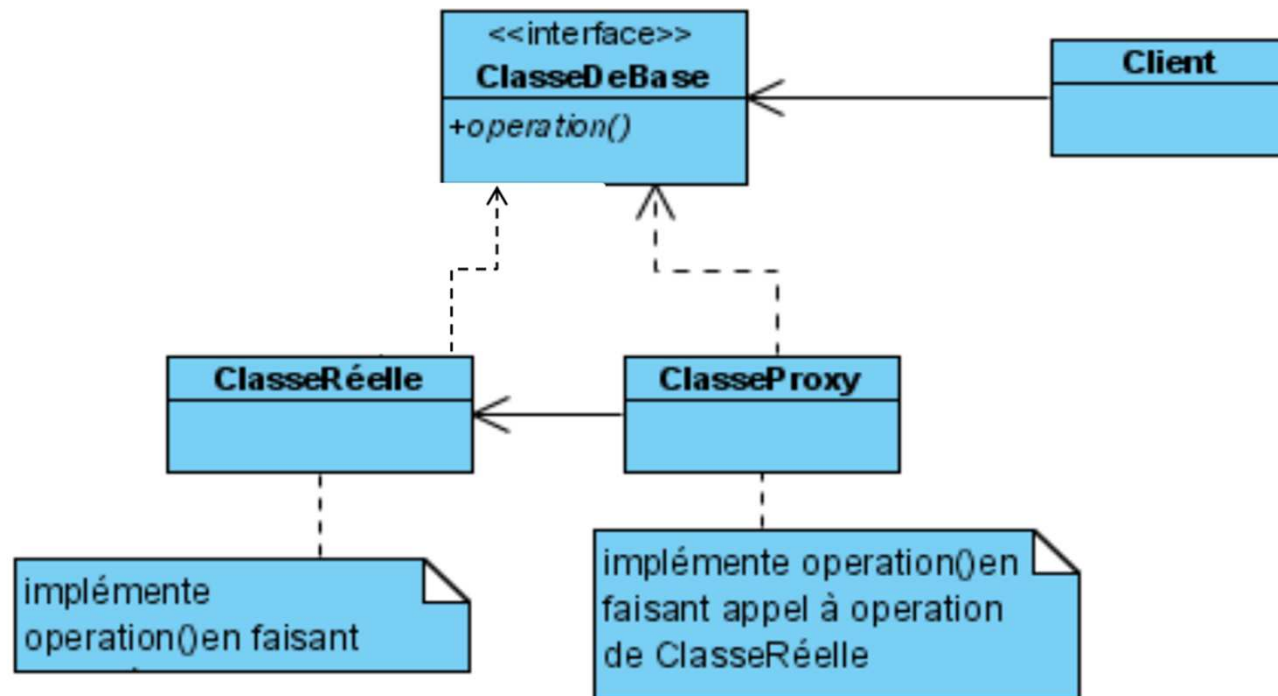
Proxy – Exemple classique

- Affichage d'une page web contenant du texte et faisant référence à d'autres objets (ex : des images) :
 - Soit afficher (donc charger) complètement un élément (texte ou image) avant d'en afficher un autre
 - Soit choisir d'afficher (donc charger) les moins volumineux et afficher les plus volumineux et/ou les plus distants par la suite
- Déroulement :
 - Le navigateur commence à afficher la page tout en téléchargeant les différentes images
 - Les images non encore chargées sont représentées par une image/symbole alternatif (carré, avec du texte, petite image temporaire...)
 - Cela revient à *donner une représentation d'un objet distant ou trop volumineux pour être mis directement en mémoire*

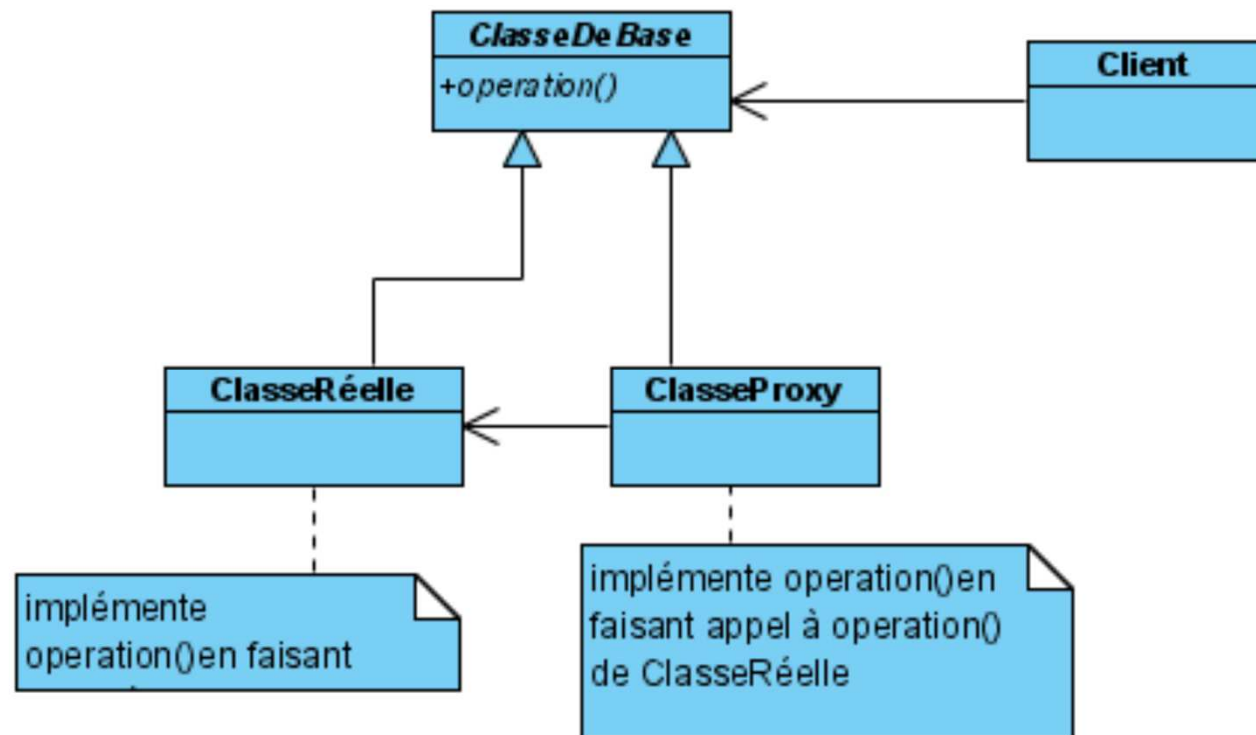
Proxy

- Assez proche de l'Adapteur, le Proxy cherche à ajouter un niveau de redirection entre l'appel d'une méthode d'un objet et l'action associée
- Mise en oeuvre : il faut
 - Une classe de base qui présente l'interface de l'objet, sans nécessairement implémenter les méthodes. Généralement, il s'agit d'une interface, parfois d'une classe abstraite
 - La classe de l'objet réel dont la construction est coûteuse. Elle dérive de la classe de base
 - La classe Proxy qui maintient une référence vers l'objet réel. Elle dérive de la classe de base
- Ce type de structure vise à pouvoir changer l'objet cible sans changer l'objet source (manipulé)

Proxy – diagramme UML



Proxy – diagramme UML

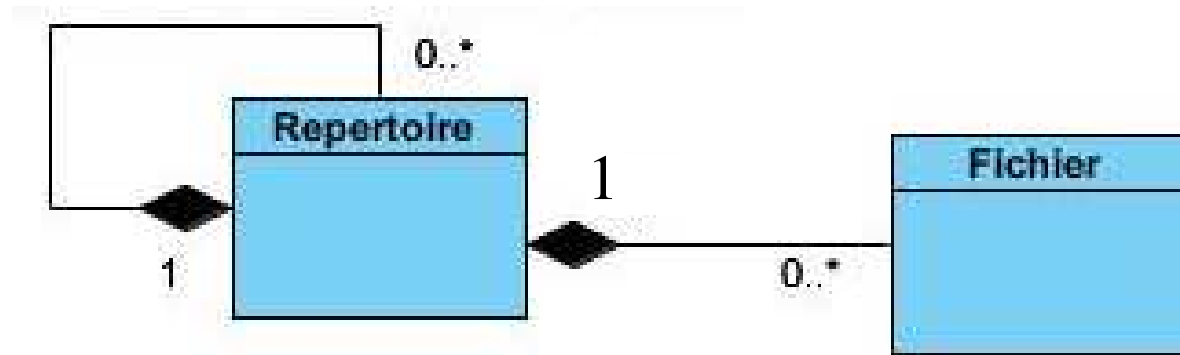


Proxy

- Lorsqu'on essaye d'accéder à la méthode *operation()* du Proxy, celui-ci vérifie si l'objet (réel !) est accessible (chargé, présent en mémoire, droits d'accès suffisants)
- Si c'est le cas, le Proxy appelle la méthode *operation()* de l'objet réel
- Sinon, le Proxy renvoie une représentation intermédiaire de l'objet ("en cours de chargement", "accès refusé", etc).

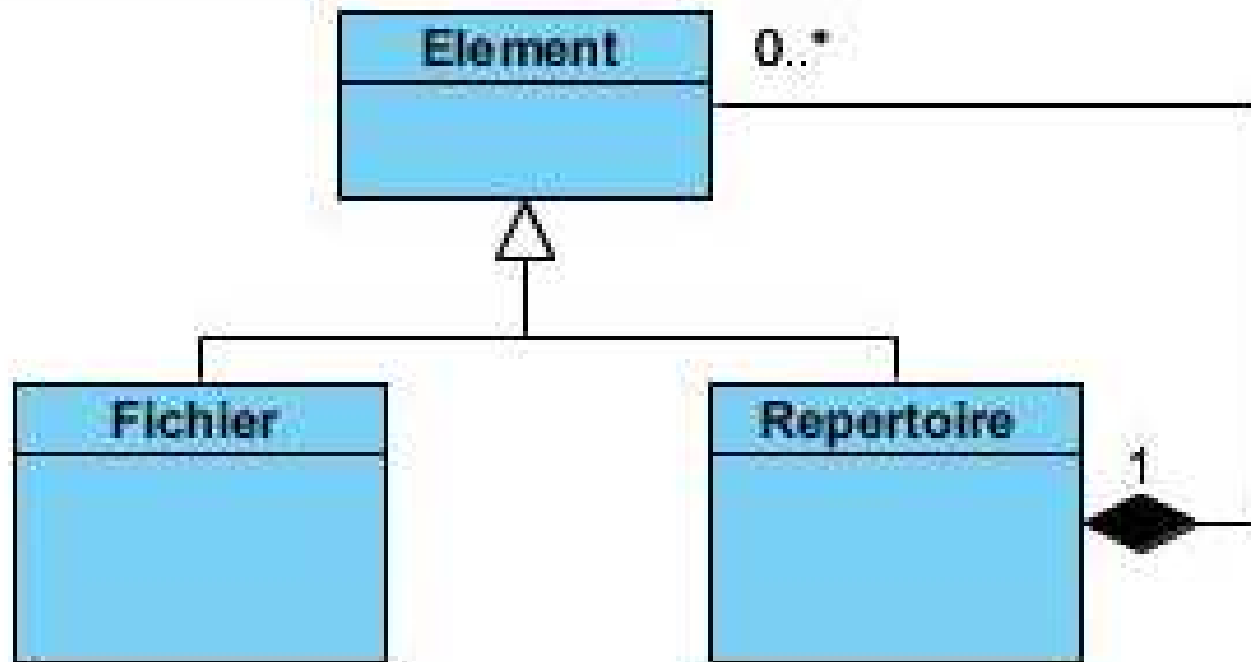
Pattern Composite

- Prenons les situations suivantes :
 - Un répertoire contient des fichiers et des répertoires.



- Quel est le cycle de vie d'un fichier et d'un sous-répertoire par rapport à leur répertoire ?

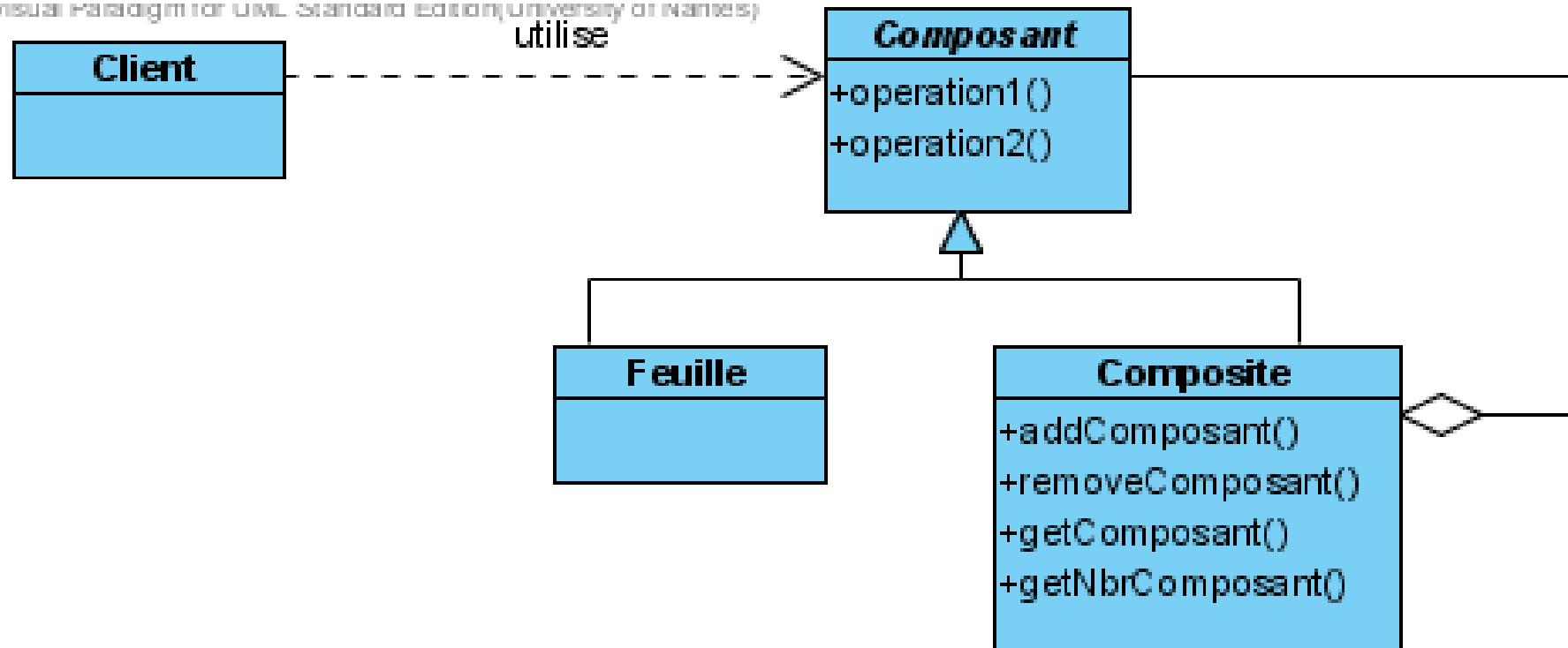
Pattern Composite



Pattern Composite

- **Objectif** : permettre un traitement uniforme des feuilles et des nœuds

visual Paradigm for UML Standard Edition (University of Nantes)



Pattern Composite

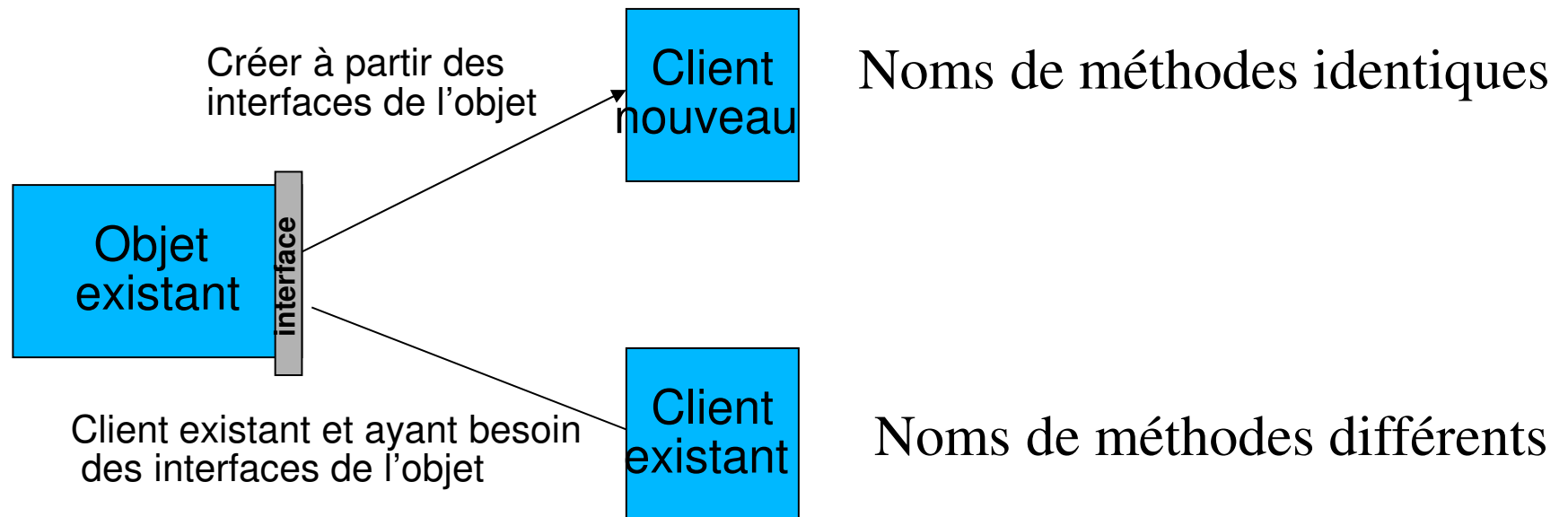
- Composite :
 - groupe d'objets contenant des éléments individuels (*feuilles*) et des éléments (*noeuds*) contenant eux-même d'autres objets.
 - maintient une collection de composants
- *Composant* : classe de base pour les objets primitifs.
- Question : quel serait le principe des opérations ?

Opérations du composite

- Les méthodes de Composite sont implémentées en itérant la collection de composants et invoquant la méthode appropriée pour chaque composant.
- Exemple :

```
// Structure d'une méthode Composite
public void draw() { // itération pour tous les composants
    for(int i=0; i < getNbComposant (); ++i) {
        // Récupère référence d'un composant et invoque sa
        // méthode
        Composant composant = getComposant(i); composant.draw();
    }
}
```

ADAPTER



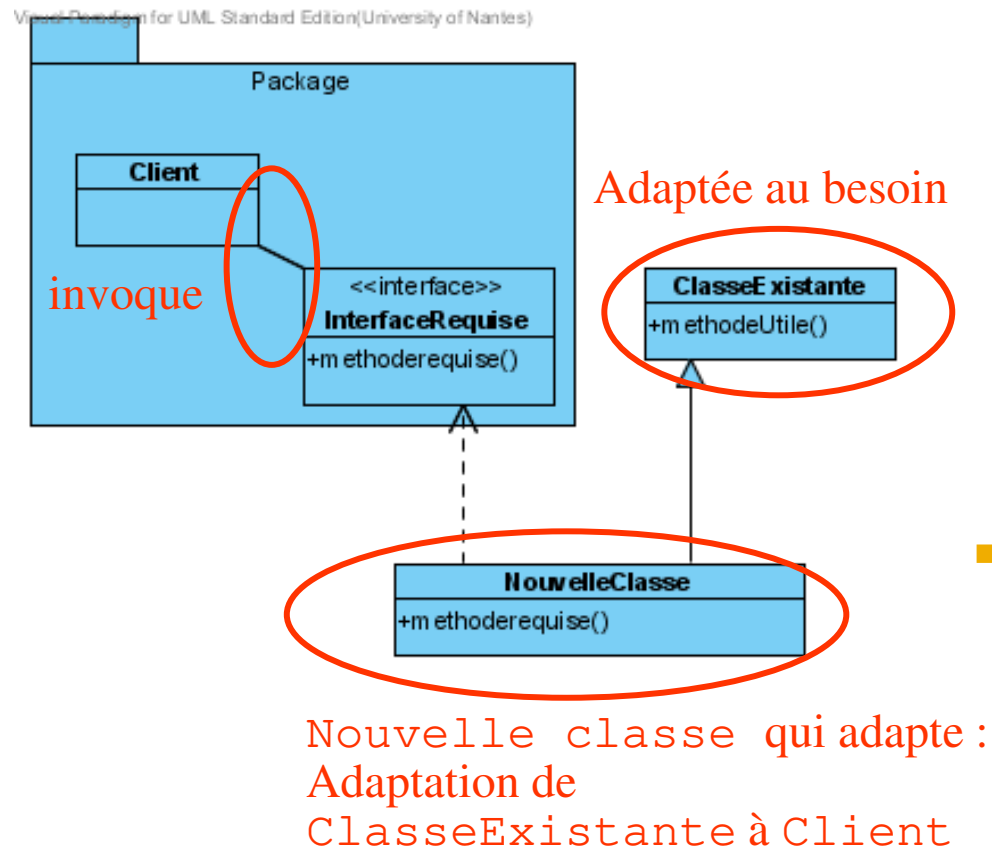
Nécessité d'adaptation

Pattern ADAPTER

ADAPTER

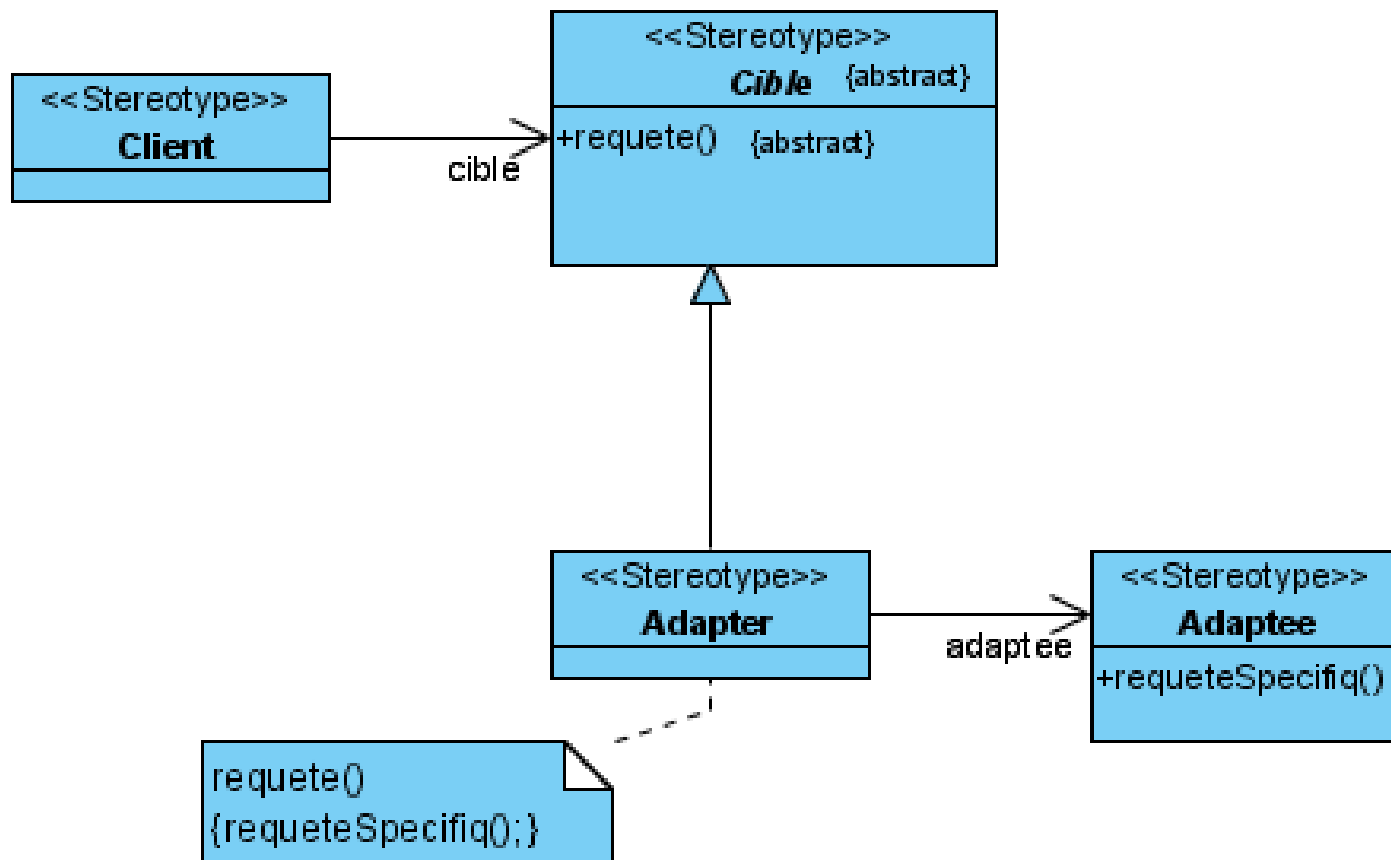
- Toute situation où l'on veut réutiliser du code, mais pas son interface de programmation.
 - Par exemple, un code ancien, ou tiers, ne respecte pas notre charte de présentation
- Créer une classe d'adaptation qui transforme les appels sur l'interface voulue en appels sur l'interface existante.
- Utilise le polymorphisme et les interfaces.

Exemple d'ADAPTER



- NouvelleClasse :
 - Étend ClasseExistante
 - Implémente InterfaceRequise
 - Redéfinit methodeRequise de manière à déléguer à methodeUtile
- Une instance de NouvelleClasse est une instance de InterfaceRequise : NouvelleClasse répond aux besoins du Client.

ADAPTER



Implémentation de la méthode abstraite en déléguant vers Adapter

Observateur - *Observer*

- Un objet O_1 sollicite un autre objet O_2 , qui lui est nécessaire à son fonctionnement : O_1 est client de O_2 .
- Si O_2 change, comment ses clients, qui en dépendent, peuvent déterminer cette situation ?
- Une solution : O_2 doit informer ses clients.
- Mauvaise solution :
 - Ce n'est pas de sa responsabilité
 - Comment savoir quels changements impactent quels clients ?

Observateur - *Observer*

- Solution :
 - établir une dépendance 1,n entre objets
 - de manière à notifier les objets dépendants lorsqu'un objet change.
- Observer permet à un objet de demander à être notifié lorsqu'un autre objet change.
- Exemple : Observer est fondamental en IHM
 - Activation d'un bouton impose à différents objets de l'application à réagir.
 - Exemple : activation du bouton 'Quitter', impose de fermer la BD puis de quitter l'application.
 - Observer est fortement présent dans Swing : *listener*

Observateur - Observer

- Observer suppose que
 - L'objet contenant les données est séparé des objets affichant ces données.

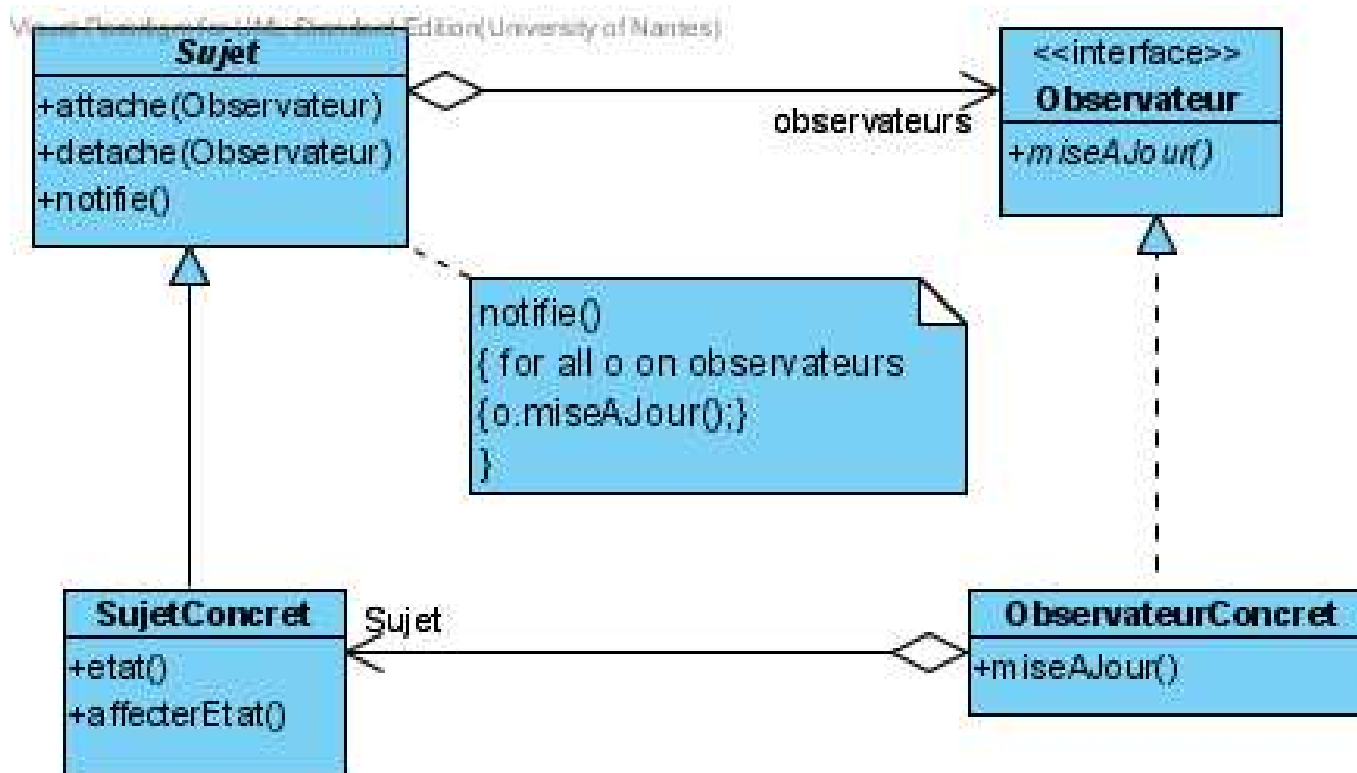


- Ces derniers *observent* les changements dans les données.
- Les données sont le *sujet*
- Les objets 'afficheurs' sont les *observateurs*

Observateur - Observer

- Chaque observateur a une interface connue que le sujet appelle quand les données changent.
- Toute entité appelée à être un sujet devra implémenter Sujet
- Toute entité appelée à être un observateur devra implémenter Observateur.

Observateur en UML



Observateur

- DP de comportement (certains le présente comme un DP de responsabilité).
- Avantage de ces interfaces : l'observateur et le sujet ne nécessitent pas de savoir ce que l'autre fait.
- Ce DP peut être plus complexe lorsque les observateurs nécessitent des données spécifiques et différentes.
 - Plutôt que de faire faire les conversions par les observateurs, il est possible, par exemple, d'utiliser le DP Adaptateur.
- Observateur et MVC peuvent aller de paire.
 - Si la vue est bien séparée du contrôleur.
 - Dans Swing, Observer n'est pas toujours couplé avec MVC. L'apparence d'un objet 'interface graphique' (ou GUI) et ses fonctionnalités sont souvent étroitement liées.

Factory - Fabrique

- DP de création
- Facette importante des systèmes : manière dont les instances sont créées.
- Utilisation systématique et directe des mécanismes d'instanciation et d'initialisation
 - créent de forts couplages
 - difficiles à maintenir au moindre changement.
- Objectif des DP de création : réduire ce couplage.
- Le DP Factory est le plus utilisé : il spécialise un objet unique à la fabrication (création) d'autres objets.



- Objectif : laisser un autre développeur définir l'interface permettant de créer un objet, tout en gardant un contrôle sur le choix de la classe à instancier.

Factory - Fabrique

- Généralement, un client qui a besoin d'un objet instancie la classe voulue via un de ses constructeurs.
- Il arrive que le client ne sache pas exactement quelle classe instancier.
- Il arrive que la manière de les instancier change.

Factory - Fabrique

■ Exemple d'Iterator :

```
public class ShowIterator
{ public static void main(String [] args)
  {List list = Arrays.asList(
                                new String[] {
                                "Fontaine", "rock","chute"});
    Iterator iter = list.iterator();
    while (iter.hasNext())
        System.out.println(iter.next());
  }
}
```

■ Quelle est la classe réelle de `iter` ?

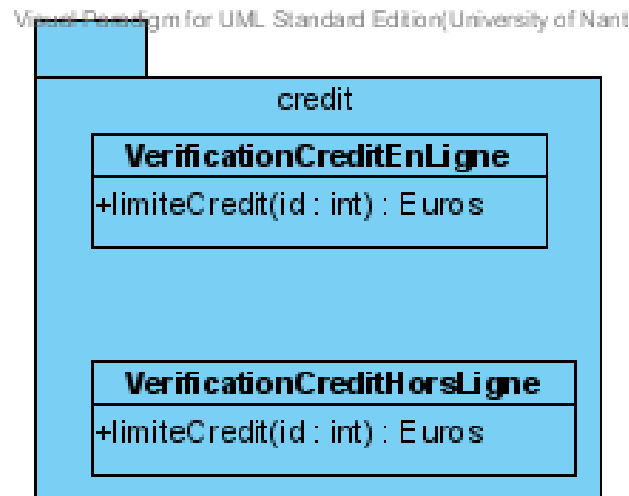
- Inutile de le savoir ...
- Si besoin, faire `iter.getClass().getName()`

Factory - Fabrique

- La fabrique rend *abstraite* au client la création et l'initialisation du produit.
- Si l'implémentation du produit change, le client reste inchangé quant à sa manière de créer le produit.
- Le client est décorrélé à la fois :
 - du produit
 - de la fabrique
- Factory permet de créer et fournit un nouvel objet tout en évitant au client de savoir quelle classe instancier.

Factory – Garder le contrôle

- Exemple : achat d'un article avec un certain montant de crédit autorisé au client. A l'achat, vérification du crédit :
 - Auprès d'un organisme de crédit
 - Décision en local si l'organisme est hors ligne

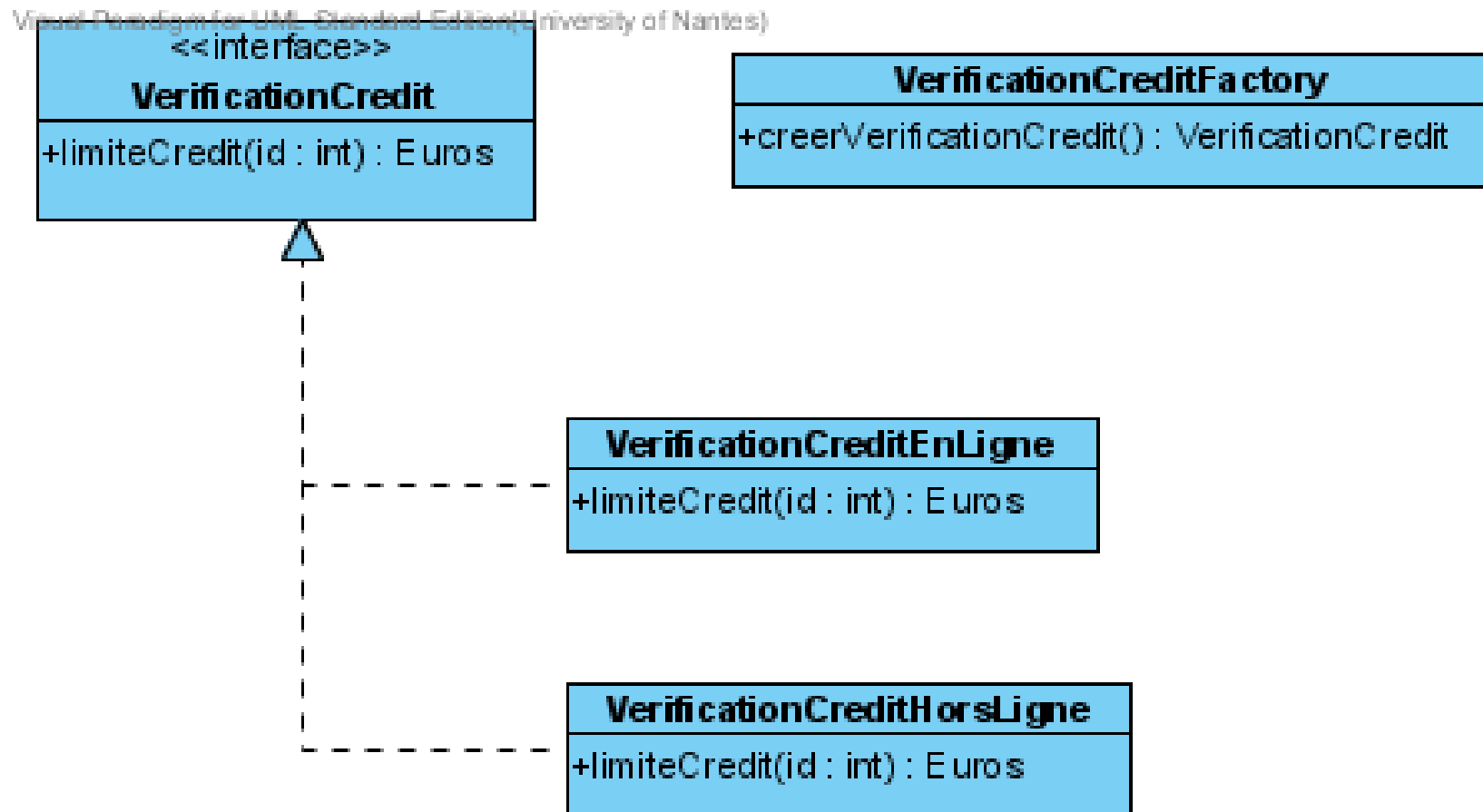


Factory – Garder le contrôle

- Quelle classe le client devra instancier ?
 - Le client ne doit pas gérer cette question.
- Utiliser une interface pour créer l'objet mais garder le contrôle sur le choix de la classe à instancier
 - Créer une interface implémentée par les deux classe.
 - Définir une classe Factory avec une méthode Factory qui retourne un objet du type de l'interface.

Factory – Garder le contrôle

- Diagramme de classes :

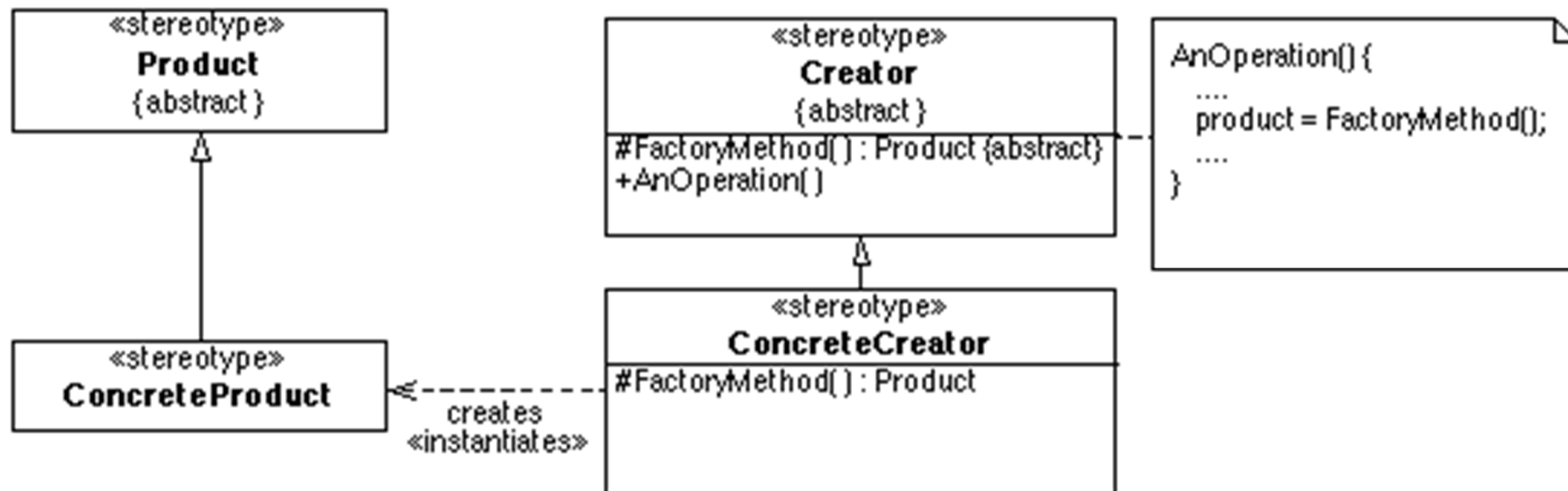


Factory – Garder le contrôle

- En supposant `agenceActive()` la méthode qui indique si l'agence est disponible, quel est le code de `creerVerificationCredit()` ?

```
public static VerificationCredit creerVerificationCredit()  
{ if (agenceActive())  
    return new VerificationCreditEnLigne();  
    return new VerificationCreditHorsLigne();  
}
```

Factory – Un diagramme UML



Singleton

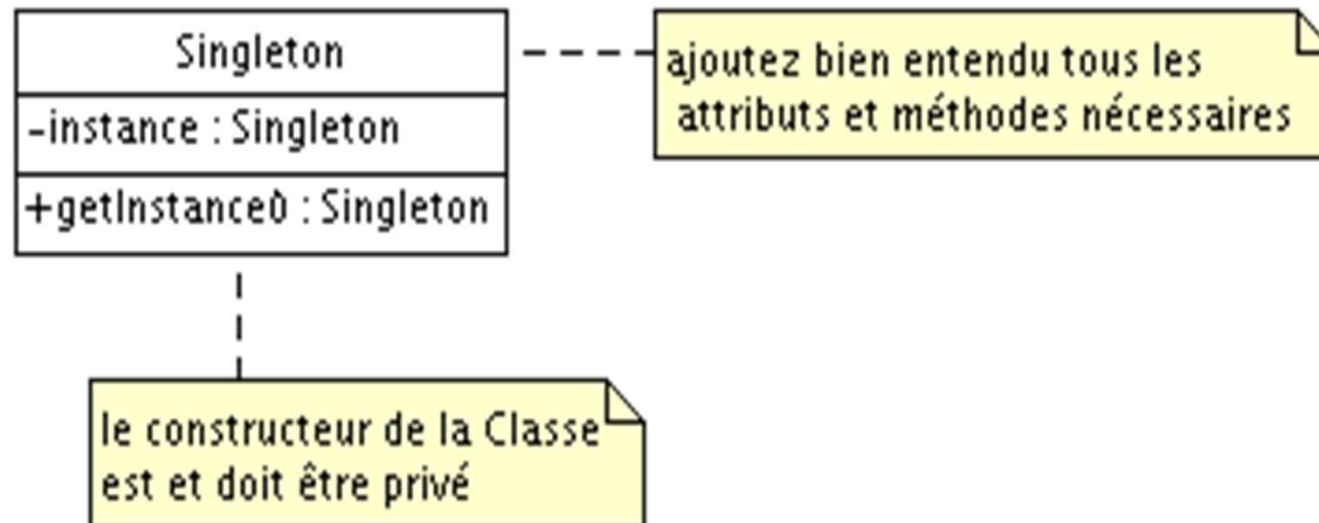
- DP de création
- Assure qu'une classe
 - a une et une seule instance
 - fournit un point d'accès global à cette instance.
- Pourquoi ?
 - Nécessaire lorsqu'un objet assume une responsabilité dont dépendent d'autres objets
 - Exemples : la fenêtre de gestion d'une application, un seul spool d'impression, un ensemble de "pool" de connexions à une BD géré par un singleton
 - Gain en mémoire mais également en sécurité (exemple spool BD : moins de connexions vers l'extérieur si la base de données n'est pas locale).

Singleton

- Une solution :
 - Une variable `static` renseignée dès que le constructeur est accédé :
`static boolean instance_flag = false;`
 - Mais n'assure en rien une création réussie d'instance.
 - Méthode pour contrôler alourdit et n'exempt pas de toute erreur
- Le DP Singleton suggère
 - de fournir une méthode publique et statique qui donne accès à l'objet singleton
 - si cette méthode crée l'objet, elle doit s'assurer qu'une seule instance pourra être créée.
 - de mettre le constructeur en privé, ou en `protected` si la classe doit être étendue.

Singleton

- Une solution



Singleton

- Il faut décider du moment de la création de l'unique instance

- En tant que champ statique dans la classe

```
private static Singleton singleton=new Singleton()
```

- Attendre le moment où elle est requise pour l'instancier :
initialisation 'paresseuse' ou *lazy-initialization* :

```
public static Singleton get Singleton()  
{if (singleton== null) singleton = new Singleton();  
// ...  
return singleton; }
```

Singleton – une implémentation

```
public class Singleton {  
  
    /** Récupère l'instance unique de la class Singleton.  
     * Remarque : le constructeur est rendu inaccessible  
     */  
  
    /** L'instance statique */  
    private static Singleton instance;  
  
    /** Constructeur redéfini comme étant privé pour interdire  
     * son appel et forcer à passer par la méthode getInstance()  
     */  
    private Singleton() { }  
  
    /** Méthode statique d'accès à l'instance et qui permet  
     * une lazy-initialization  
     */  
  
    public static Singleton getInstance() {  
        if (null == instance) { // Premier appel donc new  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Singleton

- Très utile lorsque la classe d'instance unique a des sous-classes.
 - Utilisation d'une seule définition de Singleton
 - qui peut varier en fonction d'un paramètre
 - qui permettra d'installer une seule instance précise de classe lors de l'invocation
 - une application beaucoup plus dynamique.
- Exemple de situation critique : application multi-threadée avec risque d'accès à l'initialisation du singleton par plusieurs threads. Attention à synchroniser !
- DP des plus connus mais il faut être attentif à une utilisation adéquate
 - Ne pas l'utiliser pour des classes qui utilisent énormément de données dynamiques (à l'encontre de la définition même du singleton).

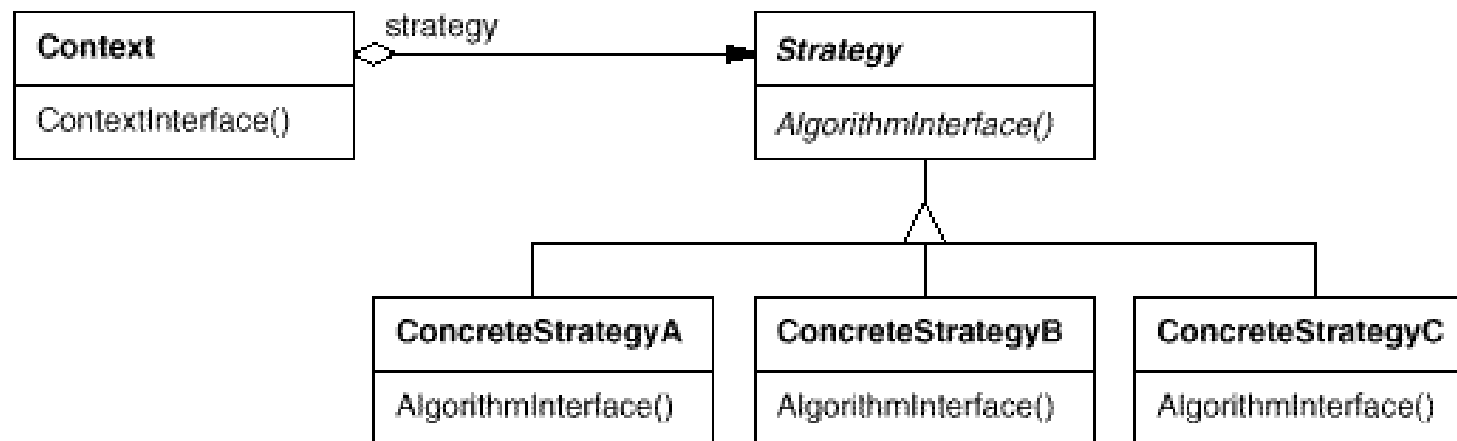
Strategy

- Le design pattern Strategy est utile pour des situations où il est nécessaire de permuter dynamiquement les algorithmes utilisés dans une application (une solution lorsqu'un problème peut être résolu par différents algorithmes).
- Il définit une famille d'algorithmes encapsulés et interchangeables. Il encapsule des approches alternatives dans des classes distinctes qui implémentent chacune une opération commune.
- Les algorithmes peuvent être sélectionnés à la volée au cours du temps d'exécution selon certaines conditions.

Strategy

- Exemples d'utilisation :
 - Plusieurs classes différant seulement dans leur comportement
 - La localisation d'applications
 - Nécessité d'avoir plusieurs variantes d'un algorithme
 - Un algorithme utilise des données que le client ne devrait pas connaître
 - Une classe définit plusieurs comportements lesquels nécessitant de nombreux tests conditionnels.
- Mise en oeuvre :
 - Définir une interface commune à tous les algorithmes
 - Implémenter les différentes stratégies concrètes, chacune implémentant l'interface prédéfinie.

Strategy – diagramme UML



Diagrammes d'états-transitions

State-chart diagrams

Diagramme d'états

- A la frontière de l'analyse et de la conception :
 - par la compréhension des cycles de vie de certains objets
 - par les méthodes qu'il permet de définir dans les classes étudiées.
- « *Le diagramme se réalise pour les classes d'objets ayant un comportement significativement complexe. Toutes les classes d'un système n'ont pas besoin d'avoir un diagramme d'état* » James RUMBAUGH

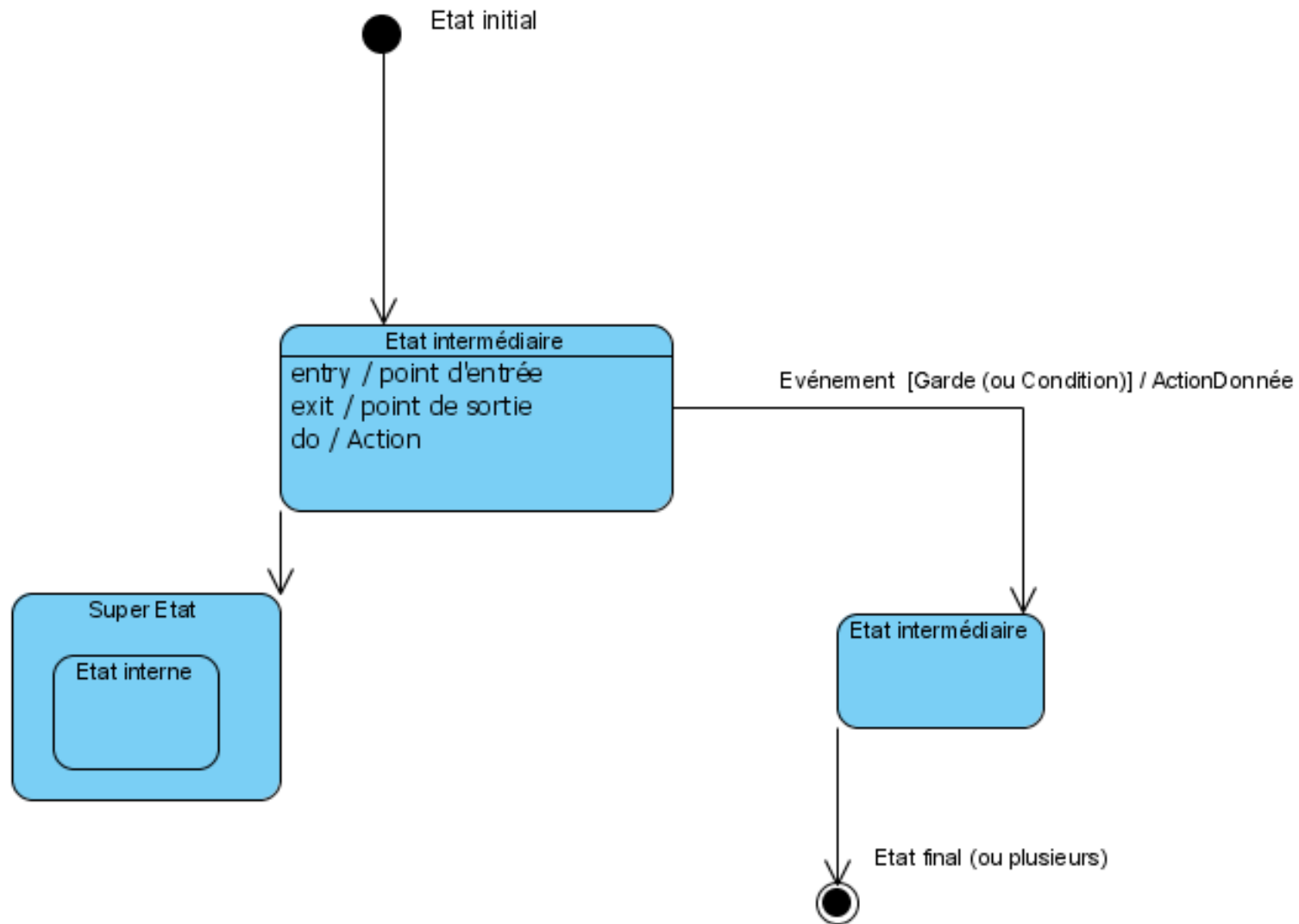
Présentation

- Automates à états finis
- Technique familière de description du comportement d'un système. En CPO, généralement :
 - Décrit le comportement et tous les états possibles des instances d'une classe tout au long du cycle de vie
 - Les transitions d'un état (de l'objet) à un autre état.
 - L'objet ne peut prendre aucun état en dehors de ceux identifiés dans le diagramme.
 - Diagramme rattaché à cette classe.

Présentation

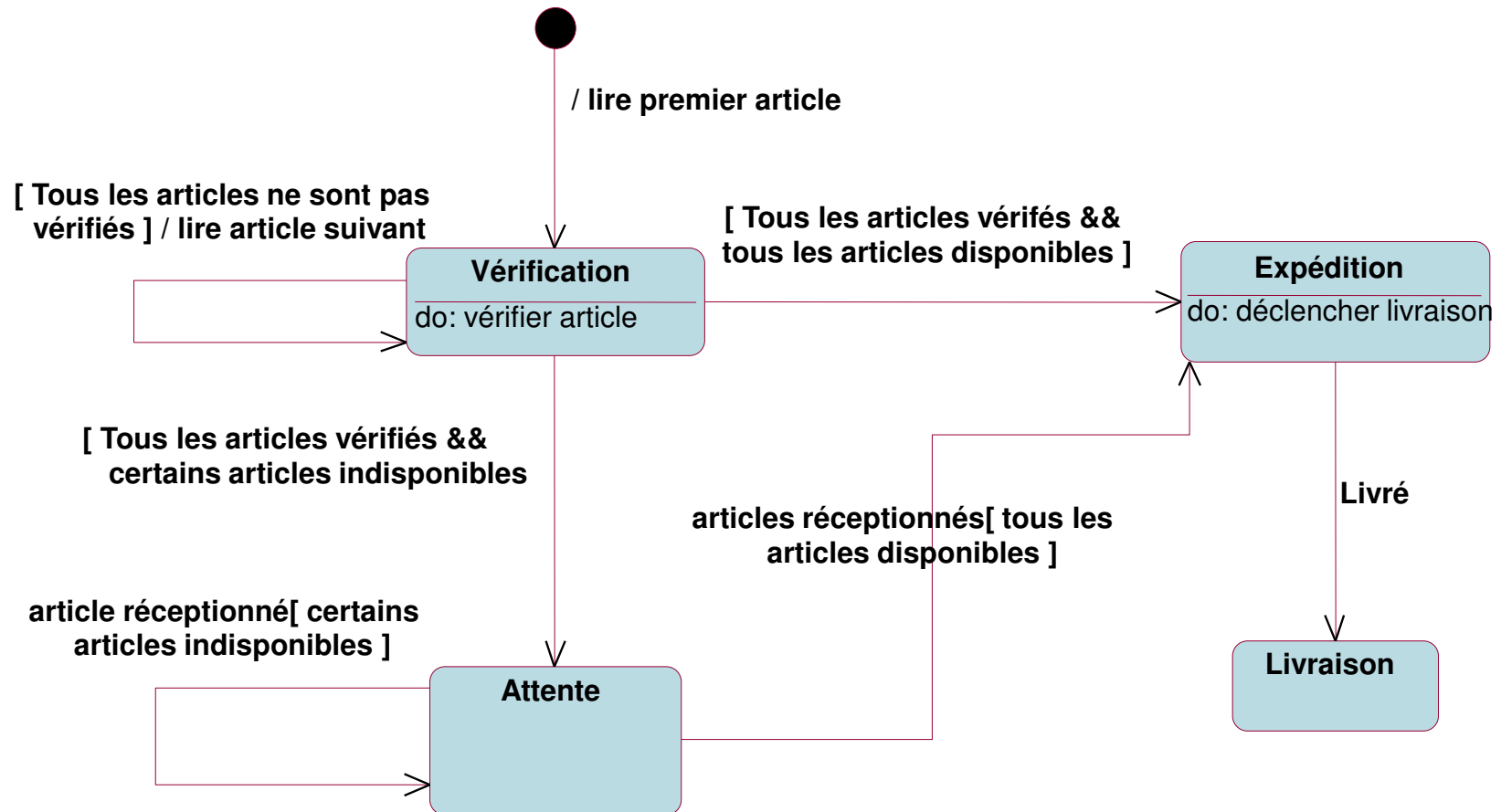
- Nécessaire pour documenter les classes ayant :
 - des états clairement identifiables
 - un comportement complexe
 - beaucoup d'événements (signaux)
- Un repère :
 - grand nombre de flux entrants et sortants de la classe au sein de diagrammes d'interactions.
- Permettent de compléter les diagrammes de classes de conception en mettant en évidence :
 - des méthodes publiques (aux actions du diagramme d'état)
 - des méthodes privées (activités du diagramme d'état)

Formalisme graphique

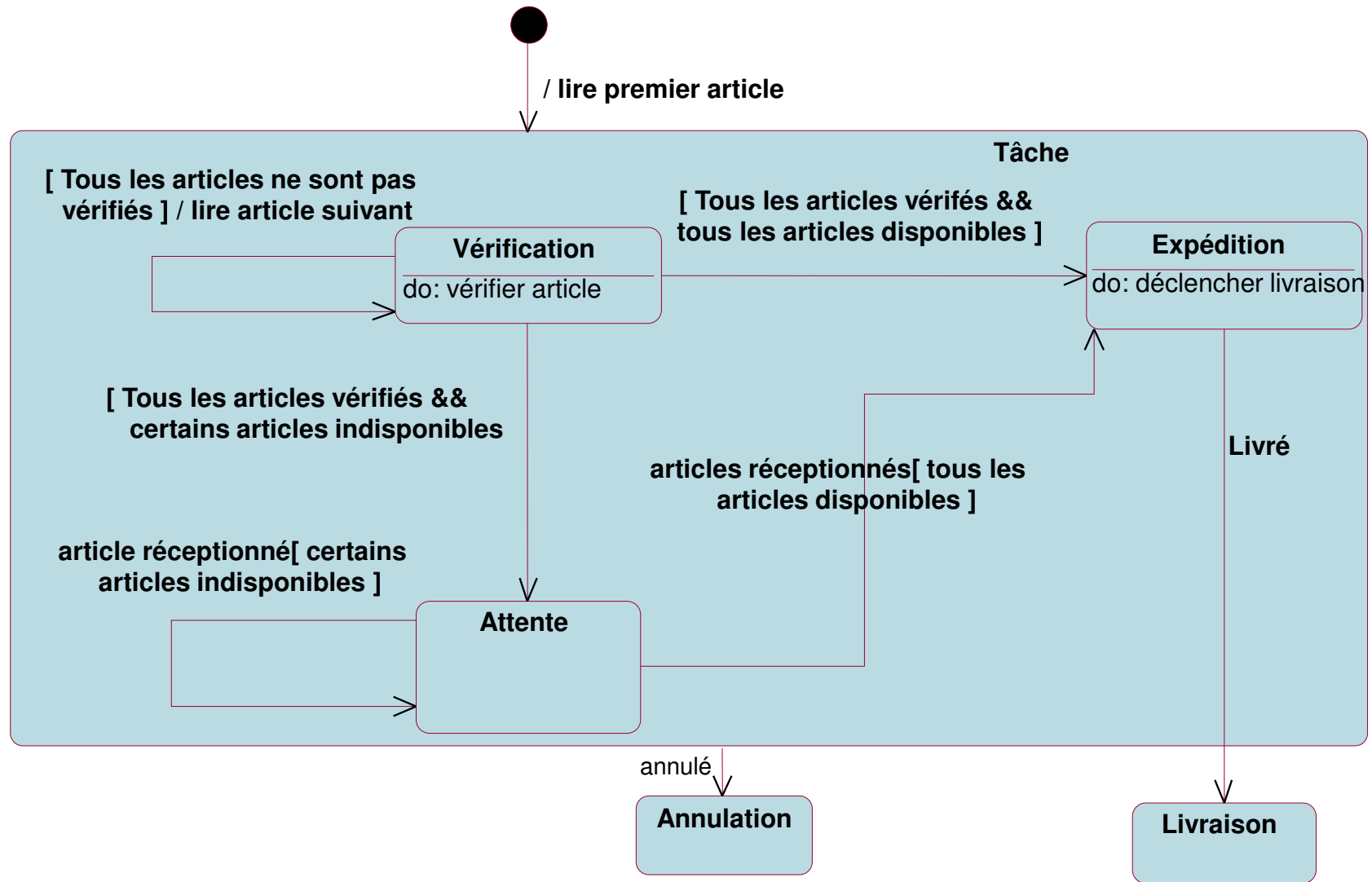


Un exemple

Les états d'un objet de la classe Commande

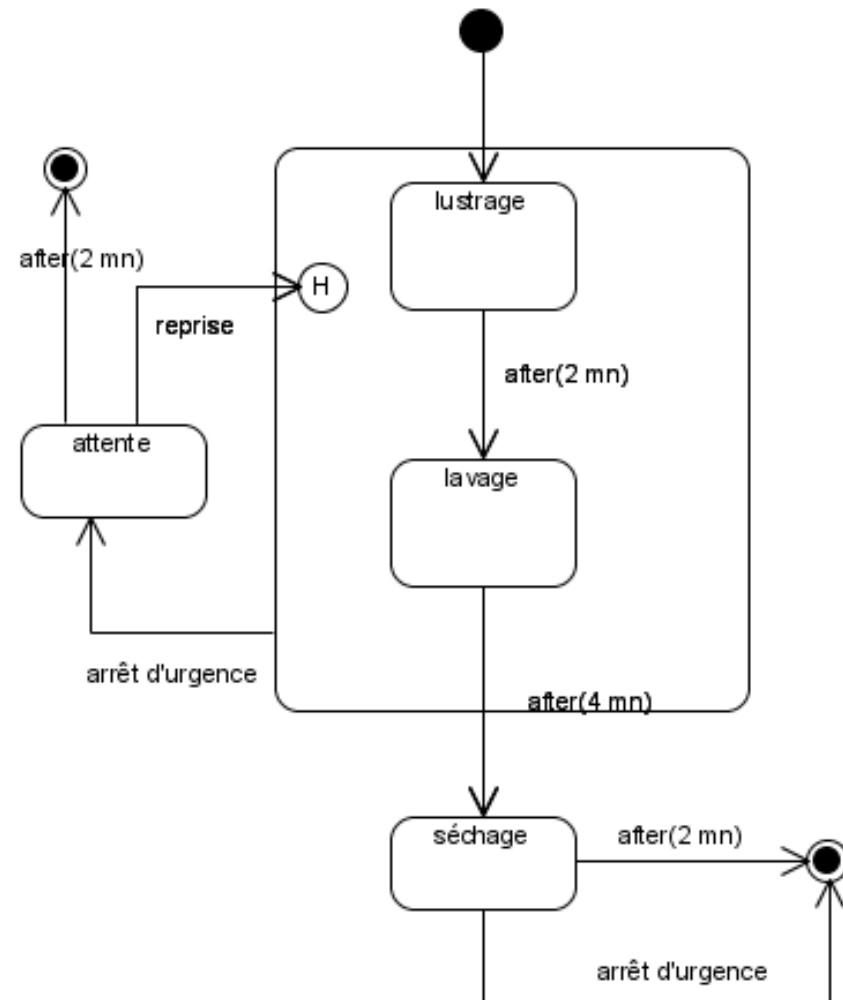


Avec super-état



Super-état

- Un **super-état** :
 - est un état qui **contient d'autres états**
 - a un **historique**, *càd* une mémoire du dernier sous-état actif.

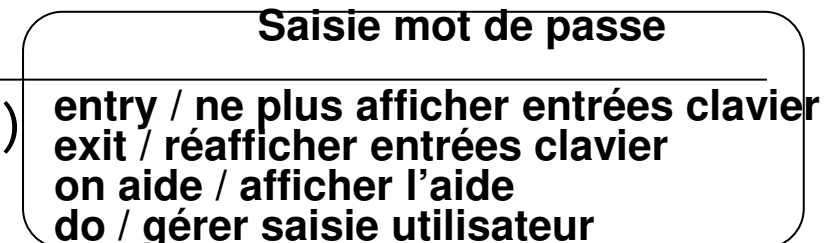


Événement/action

- Pour lier un événement à une action
 - syntaxe : événement/action.
 - Une action se réalise de façon instantanée
- call action : lorsqu'une transition se produit

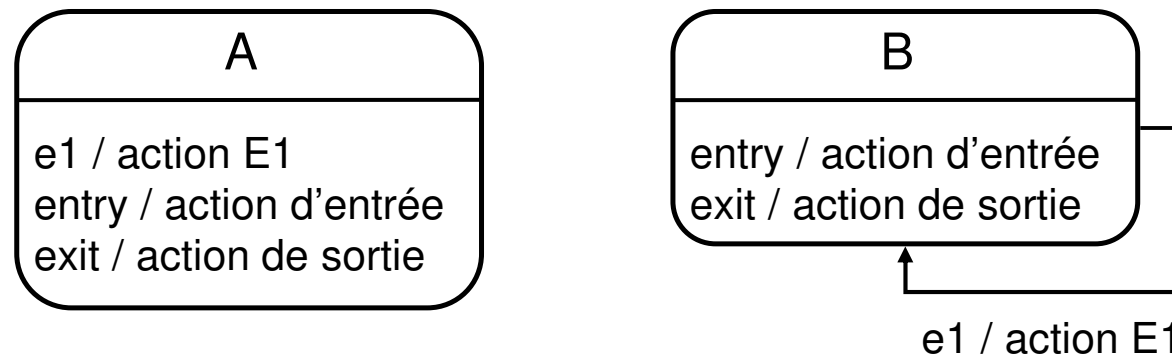


- actions internes : des méthodes prédéfinies et de l'utilisateur permettent de le faire
 - entry (exécution à l'entrée)
 - do(action récurrente dans l'état)
 - on <evenement>(si événement valide alors exécution)
 - exit(exécution à la sortie)



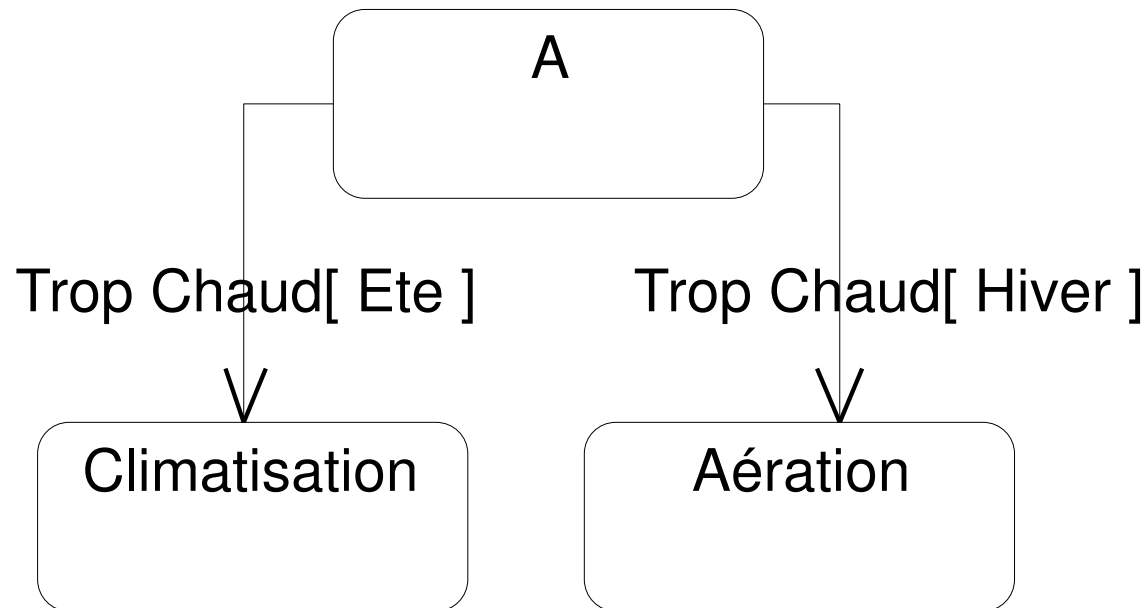
Actions internes

- Différence fondamentale entre action interne et transition réflexive
 - l'action interne n'entraîne pas l'exécution des actions d'entrée et de sortie



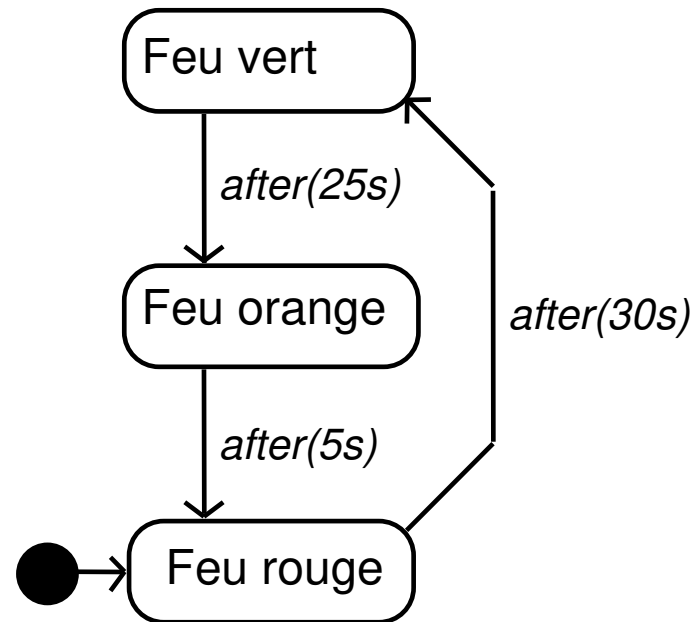
Concept *garde*

- une garde (*guard*) est une condition qui doit être satisfaite pour que la transition ait lieu



Exemple

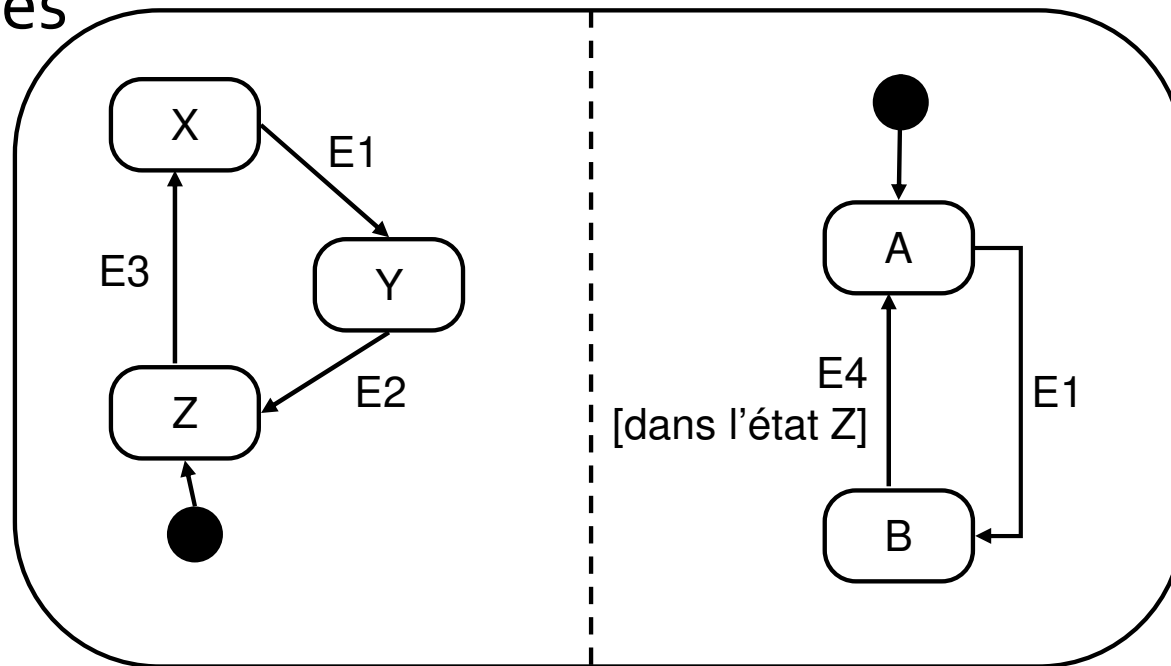
Feux de signalisation simplifié



États concurrents

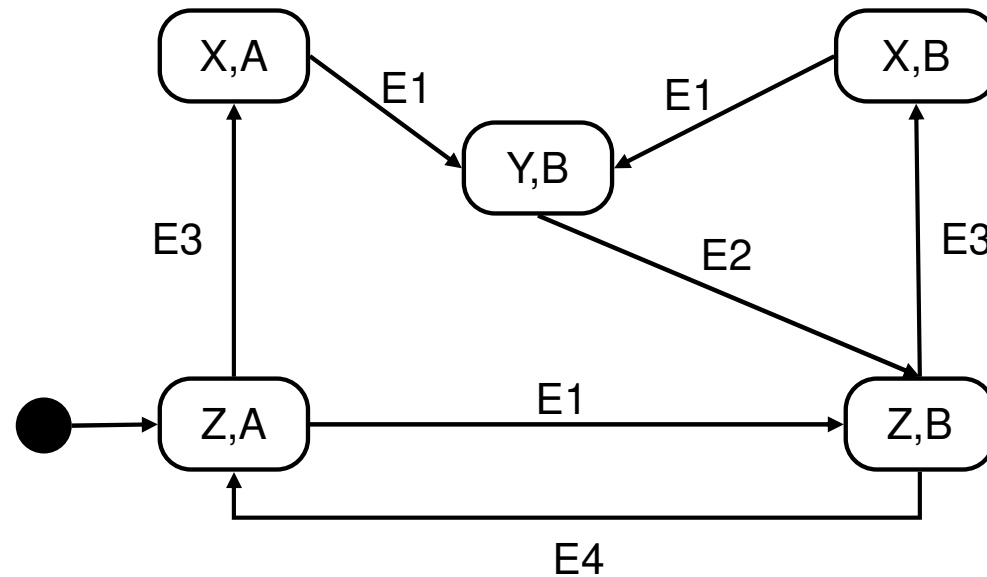
- On objet peut être simultanément dans différents sous-états
 - représente une forme de parallélisme entre automates

- Exemple



États concurrents

- Equivalent à :



Utilisation

- Cycle de vie des objets d'une même classe.
- Donc son comportement au travers des différents cas d'utilisation.
- Ne conviennent pas pour la description de collaborations entre objets.
- Utilité de combiner avec d'autres techniques: les diagrammes d'interactions.
- Ne les utiliser que pour les cycles de vie complexes.